Temporal with Node.js

The Complete Practical Tutorial

From installation and project setup, through SDK imports and the runtime model and 15 tested real-world examples, to signals, timers, error handling, versioning, testing, production deployment and a complete end-to-end order-processing service.

Verified with @temporalio/* 1.24 · Temporal CLI 1.9 · Node.js 24 · TypeScript 7

Contents

Part I — Getting Started 3

1. Why Temporal? . . . 3
2. Architecture & Runtime Model . . . 4
3. Installation . . . 5
4. Project Setup from Scratch . . . 7
5. SDK Packages, Imports & the Workflow Sandbox . . . 9

Part II — Core Building Blocks 10

6. Activities . . . 10
7. Workflows . . . 16
8. Workers . . . 19
9. Clients . . . 20
10. First Run: Hello World End to End . . . 21

Part III — Workflow Features 23

11. Signals, Queries & Updates . . . 23
12. Timers & Schedules . . . 28
13. Error Handling, Retries & Timeouts . . . 34
14. Cancellation . . . 37
15. Child Workflows, Continue-As-New & Fan-out . . . 41

Part IV — Common Patterns 50

16. Patterns Cookbook . . . 50

Part V — Production Readiness 57

17. Versioning Workflow Code . . . 57
18. Testing . . . 62
19. Observability & Debugging . . . 65
20. Deployment . . . 66

Part VI — End-to-End Project: Order Service 69

21. Building an Order-Processing Service . . . 69

Appendices 76

A. CLI Reference . . . 76
B. Troubleshooting Quick Reference . . . 77
C. Glossary . . . 78

Part I — Getting Started

1. Why Temporal?

Most backend processes are more than one step: charge a card, reserve stock, send an email, wait for approval, ship. Coding that reliably by hand means queues, cron jobs, status columns, retry tables, dead-letter handling and idempotency checks — and still hoping nothing crashes between step 2 and step 3.

Temporal is a **durable execution** platform. You write the process as plain async TypeScript. Temporal records every step in an **event history**; if your process crashes, a new worker picks up the history, replays the code to the exact same point, and continues. Waiting 30 days is one line of code and costs nothing while waiting.

Without Temporal	With Temporal
Hand-written retry loops, backoff, DLQs	Declarative retry policy per activity
State stored in DB status columns	State is just local variables in the workflow
Cron + polling to resume long waits	`await sleep('30 days')`
Custom compensation / rollback logic spread across services	Saga pattern in one function with try/catch
Hard to see where a process is stuck	Web UI shows full event history of each run

Common use cases: order & payment flows, onboarding, subscriptions/billing, data pipelines & ETL, infrastructure provisioning, human approvals, AI agent orchestration, and any microservice orchestration that must not lose progress.

1.1 Real-world examples in this guide

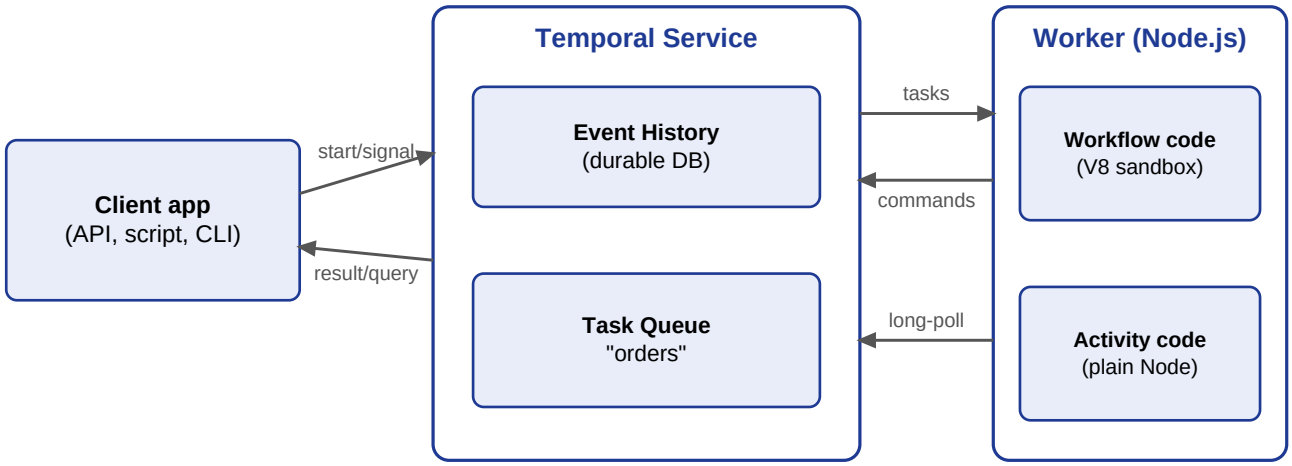
Every topic is shown twice: first the concept, then a **real-world example** that runs and is covered by tests. All examples live in `temporal-examples/` next to this PDF (35 tests, all passing).

Scenario	Topics	Section
Charging a card	Activities, retries, idempotency, typed failures	6.5
Flight search	Parallel activities, local activities, partial failure	7.5
Shopping cart	Signals, queries, updates, validators, idle timeout	11.1
14-day free trial	Durable timers, condition + timeout	12.3
Nightly sales report	Schedules, time zones, overlap/catch-up	12.4
Loan application	Error types, timeouts, local activities	13.5
Video transcoding	Heartbeats, resume, cancellation, cleanup	14.1
Employee onboarding	Child workflows, parent close policy	15.4
Loyalty account	Entity workflow, continue-as-new, dedupe	15.5
E-mail campaign	Bounded fan-out, pagination, progress query	15.6
Trip booking	Saga compensation	16.1
Expense approval	Human-in-the-loop, escalation	16.2

Scenario	Topics	Section
Identity verification	Polling via retry policy	16.3
Document review	Async activity completion (webhooks)	16.4
Live fraud-check rollout	Versioning, patching, replay tests	17.3

2. Architecture & Runtime Model

A Temporal application has three parts: the **Temporal Service** (server), your **Workers** (Node.js processes running your code) and **Clients** (code that starts and talks to workflows). The server *never* runs your code — it stores history and hands out tasks; workers pull tasks, execute them and report back.



2.1 Key terms

Term	Meaning
Workflow Definition	An exported async function in `workflows.ts`. Deterministic orchestration logic.
Workflow Execution	One running instance, identified by **Workflow ID** + **Run ID**.
Activity	A normal async function with side effects (HTTP, DB, files, email). Retried automatically.
Worker	Process that polls a task queue and runs workflow + activity code.
Task Queue	Named queue. Clients start workflows on it; workers listen on it.
Namespace	Isolation unit (like a database). Dev server default: `default`.
Event History	Append-only log of everything that happened in an execution.
Workflow Task	"Advance the workflow code" job given to a worker.
Activity Task	"Run this activity with these args" job given to a worker.
Command	What workflow code asks for: schedule activity, start timer, complete, etc.

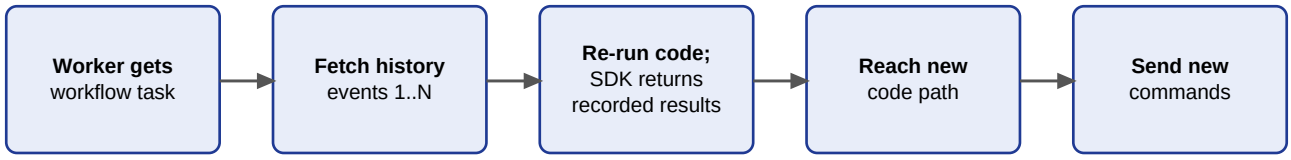
2.2 What happens at runtime (step by step)

1. Client calls `client.workflow.start(orderWorkflow, {...})`. The server writes *WorkflowExecutionStarted* and puts a **workflow task** on the task queue.

2. A worker long-polling that queue receives the task, loads the workflow bundle into a V8 isolate, and runs the function until it blocks on something (e.g. an activity call).
3. The worker returns **commands** ("schedule activity chargeCard") to the server, which records them and places an **activity task** on the queue.
4. A worker (same or different) picks up the activity task, runs your real Node function, and reports the result or error. The server records *ActivityTaskCompleted*.
5. The server schedules a new workflow task. The worker feeds the result into the waiting `await` and the workflow continues to its next step.
6. When the function returns, the worker sends *CompleteWorkflowExecution*. The result is stored and returned to anyone awaiting `handle.result()`.

2.3 Replay: how crashes are survived

Workers cache running workflows in memory. If a worker dies (or the cache is evicted), the next worker receives the full event history and **replays** the workflow code from the beginning. Every `await` on an activity that already completed resolves instantly with the recorded result, so the code arrives at the same state without re-running side effects.



Replay: completed activities are NOT re-executed; their results come from history.

This is why workflow code must be **deterministic**: given the same history, it must issue the same commands in the same order. Anything non-deterministic (I/O, random, wall-clock time) belongs in activities. The SDK helps by sandboxing workflow code (Chapter 5).

3. Installation

3.1 Prerequisites

- **Node.js 20 or newer** (examples verified on Node 24) — check with `node -v`.
- npm / pnpm / yarn.
- Supported OS: Windows x64, macOS (Intel/Apple Silicon), Linux x64/arm64 with glibc. The worker uses a native Rust core, so **Alpine/musl images are not supported**.
- Optional: Docker (for a full self-hosted server), Git.

3.1a Tested with

Every command and the complete Part VI project in this guide were run against these versions (September 2026). The companion projects are `temporal-order-service/` (Part VI) and `temporal-examples/` (15 real-world examples, 35 tests), next to this PDF.

Component	Version	Notes
Node.js	24.15	20+ supported
`@temporalio/*` (client, worker, workflow, activity, testing)	1.24.0	keep all SDK packages on the same version
Temporal CLI / Server / UI	1.9.1 / 1.32.0 / 2.54.1	`temporal server start-dev`
TypeScript	7.0.2	native compiler: use `tsx`, not ts-node/ts-jest
`tsx`	4.23	runs .ts files, watch mode
Express	5.2.1	async errors handled natively
OS	Windows 11	also works on macOS / Linux (glibc)

3.2 Install the Temporal CLI

The CLI contains a complete single-binary dev server, the Web UI and admin commands.

```
# macOS (Homebrew)
brew install temporal

# macOS / Linux (install script)
curl -sSf https://temporal.download/cli.sh | sh
# adds ~/.temporalio/bin - add it to your PATH

# Windows
#  1. Download the Windows amd64 zip from https://temporal.io/download
#  2. Extract temporal.exe to e.g. C:\tools\temporal
#  3. Add that folder to PATH (System Properties > Environment Variables)

# Verify
temporal --version
```

3.3 Start the development server

```
# In-memory (data lost on exit) - fastest for learning
temporal server start-dev

# Persist to a SQLite file, custom UI port, pre-create a namespace
temporal server start-dev --db-filename temporal.db --ui-port 8233 --namespace orders
```

Endpoint	Default	Used by
gRPC frontend	`localhost:7233`	Workers and Clients
Web UI	`http://localhost:8233`	You, in the browser
Namespace	`default`	Pass `namespace` in Worker/Client options

3.4 Other server options

- **Docker Compose** (Postgres/MySQL/Elasticsearch-backed, closer to production): clone `github.com/temporalio/docker-compose` and run `docker compose up`.
- **Kubernetes**: official Helm charts (`temporalio/helm-charts`).
- **Temporal Cloud**: managed service; you get a namespace endpoint plus an API key or mTLS certificates (Chapter 20).

3.4a No CLI? Start the dev server from Node

The testing package can download and run the same dev server binary for you — handy on locked-down machines or in CI (verified on Windows):

```
// scripts/dev-server.ts   ->   npx tsx scripts/dev-server.ts
import { TestWorkflowEnvironment } from '@temporalio/testing';

async function main() {
  const env = await TestWorkflowEnvironment.createLocal({
    server: { ip: '127.0.0.1', port: 7233, ui: true, uiPort: 8233 },
  });
  console.log('Temporal dev server on localhost:7233, UI on http://localhost:8233');
  const stop = async () => { await env.teardown(); process.exit(0); };
  process.on('SIGINT', stop);
  process.on('SIGTERM', stop);
  setInterval(() => {}, 1 << 30);   // keep process alive
}
main().catch((err) => { console.error(err); process.exit(1); });
```

3.5 Useful admin commands

```
temporal operator namespace create --namespace orders
temporal operator namespace list
temporal operator search-attribute create --name CustomerId --type Keyword
temporal operator cluster health
```

4. Project Setup from Scratch

4.1 Option A: scaffold a sample

```
npx @temporalio/create@latest ./my-temporal-app --sample hello-world
cd my-temporal-app
npm run start.watch     # terminal 1: worker with auto-reload
npm run workflow        # terminal 2: runs the client
```

Other useful samples: `activities-cancellation-heartbeating`, `child-workflows`, `saga`, `schedules`, `expense`, `nextjs-ecommerce-oneclick`. List them all at github.com/temporalio/samples-typescript.

4.2 Option B: manual setup (recommended to learn)

```
mkdir temporal-orders && cd temporal-orders
npm init -y

# Runtime SDK packages
npm install @temporalio/client @temporalio/worker @temporalio/workflow \
            @temporalio/activity @temporalio/common

# Dev tooling
npm install -D typescript tsx @types/node @temporalio/testing
npx tsc --init
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "sourceMap": true,
    "rootDir": "./src",
    "outDir": "./lib"
  },
  "include": ["src/**/*.ts"]
}
```

package.json (scripts section)

```
{
  "name": "temporal-orders",
  "private": true,
  "scripts": {
    "build": "tsc",
    "dev-server": "tsx scripts/dev-server.ts",
    "worker": "tsx src/worker.ts",
    "worker:watch": "tsx watch src/worker.ts",
    "client": "tsx src/client.ts",
    "api": "tsx src/api.ts",
    "start:worker": "node lib/worker.js",
    "test": "node --import tsx --test test/*.test.ts"
  }
}
```

Watch out: TypeScript 7 is a native (Go) compiler with no JavaScript API, so `ts-node`, `ts-jest` and similar tools fail on a fresh install. Use `tsx` to run TypeScript and Node's built-in test runner (or Vitest) for tests. TS 7 also removed `"moduleResolution": "node"` — use `"nodenext"`.

4.3 Recommended layout

```
temporal-orders/
  src/
    shared.ts        # task queue name, shared types (imported by all)
    activities.ts    # side effects - runs in normal Node.js
    workflows.ts     # orchestration - bundled & run in V8 sandbox
    worker.ts        # registers workflows + activities, polls task queue
    client.ts        # starts workflows / sends signals (scripts, APIs)
    api.ts           # optional HTTP layer (Express) that uses the client
  test/
    workflows.test.ts
  tsconfig.json
  package.json
```

4.4 Plain JavaScript and ES modules

TypeScript is optional. In JavaScript, drop the types and keep everything else the same (workflow files can be `.js`). With ES modules (`"type": "module"`), `require.resolve` is unavailable — resolve the workflows file like this:

```
// worker.mjs
import { fileURLToPath } from 'node:url';
const workflowsPath = fileURLToPath(new URL('./workflows.js', import.meta.url));
```

5. SDK Packages, Imports & the Workflow Sandbox

5.1 Which package goes where

Package	Import it in	Main exports
`@temporalio/workflow`	workflows.ts ONLY	proxyActivities, proxyLocalActivities, sleep, condition, defineSignal/Query/Update, setHandler, executeChild, startChild, continueAsNew, patched, CancellationScope, workflowInfo, uuid4, log, ApplicationFailure
`@temporalio/activity`	activities.ts	activityInfo, heartbeat, sleep, cancelled, cancellationSignal, log, Context, ApplicationFailure
`@temporalio/worker`	worker.ts	Worker, NativeConnection, Runtime, DefaultLogger, bundleWorkflowCode
`@temporalio/client`	client.ts, api.ts	Connection, Client, WorkflowHandle, WorkflowNotFoundError, WorkflowFailedError, ScheduleOverlapPolicy
`@temporalio/common`	anywhere	ApplicationFailure, failure types, RetryPolicy type, data converter types
`@temporalio/testing`	tests	TestWorkflowEnvironment, MockActivityEnvironment

5.2 The import rules

```
// workflows.ts
import { proxyActivities, sleep } from '@temporalio/workflow';  // OK
import type * as activities from './activities';               // OK - type only
import { TASK_QUEUE, type OrderInput } from './shared';        // OK - pure code
import { formatMoney } from './utils/money';                   // OK if deterministic

import * as activities from './activities';    // BAD: pulls DB/HTTP libs into sandbox
import axios from 'axios';                     // BAD: network I/O
import fs from 'fs';                           // BAD: Node built-in, bundling fails
import { Client } from '@temporalio/client';   // BAD: not allowed in workflows
```

Why `import type`? The workflow only needs the *signatures* of your activities to create typed proxies; the real implementations are registered on the Worker. Type-only imports disappear at compile time, so nothing leaks into the bundle.

5.3 What the sandbox does for you

When a Worker starts, it uses webpack to **bundle** `workflowsPath` and all its imports, then runs each workflow in an isolated V8 context where:

- `Math.random()` is seeded deterministically per execution.
- `Date.now()` / `new Date()` return the time of the current workflow task (constant during a task, replayed identically).
- `setTimeout` is replaced by a durable timer (prefer `sleep()`).
- Node built-ins (`fs`, `http`, `process`) are unavailable — bundling fails fast if you import them.
- Global state is isolated per execution.

Watch out: The sandbox cannot protect you from logic changes. Changing the order or number of activity calls in a deployed workflow can break running executions on replay. Use versioning (Chapter 17).

Part II — Core Building Blocks

6. Activities

Activities are ordinary async functions. They can do anything: call APIs, query databases, read files, use any npm package. They should be **idempotent** because Temporal may run them more than once (retries, worker crash after completion but before reporting).

6.1 Basic activities

```
// src/activities.ts
import { log } from '@temporalio/activity';

export async function sendEmail(to: string, subject: string): Promise<void> {
  log.info('Sending email', { to, subject });
  await mailer.send({ to, subject });
}

export async function getExchangeRate(from: string, to: string): Promise<number> {
  const res = await fetch(`https://api.example.com/rate?from=${from}&to=${to}`);
  if (!res.ok) throw new Error(`Rate API ${res.status}`);   // retried automatically
  return (await res.json()).rate;
}
```

6.2 Dependency injection (DB clients, config)

Create activities from a factory so they share connections and are easy to test:

```
// src/activities.ts
import type { Pool } from 'pg';

export function createActivities(db: Pool) {
  return {
    async saveOrder(orderId: string, total: number): Promise<void> {
      await db.query(
        'INSERT INTO orders(id,total) VALUES($1,$2) ON CONFLICT (id) DO NOTHING',
        [orderId, total],            // ON CONFLICT = idempotent on retries
      );
    },
    async getOrderTotal(orderId: string): Promise<number> {
      const r = await db.query('SELECT total FROM orders WHERE id=$1', [orderId]);
      return Number(r.rows[0].total);
    },
  };
}

// worker.ts:  activities: createActivities(new Pool({ connectionString: ... }))
// workflows.ts:
//   proxyActivities<ReturnType<typeof createActivities>>({ ... })
```

6.3 Activity context

```
import { activityInfo, heartbeat, sleep, cancellationSignal, log }
  from '@temporalio/activity';

export async function processVideo(url: string): Promise<string> {
  const info = activityInfo();
  log.info('Starting', {
    attempt: info.attempt,
    workflowId: info.workflowExecution?.workflowId,   // undefined outside workflows
  });

  const res = await fetch(url, { signal: cancellationSignal() }); // aborts on cancel
  for (let pct = 0; pct <= 100; pct += 10) {
    heartbeat(pct);            // tells server "still alive" + progress
    await sleep('2 seconds');  // cancellation-aware sleep
  }
  return 'done';
}
```

activityInfo() field	Use
`attempt`	Current attempt number (starts at 1).
`activityId / activityType`	Identify the call in logs.
`workflowExecution`	{ workflowId, runId } of the calling workflow.
`heartbeatDetails`	Last value passed to heartbeat() before a retry — resume from checkpoint.
`taskToken`	Needed for asynchronous completion (see below).

6.4 Asynchronous activity completion

When the real work is finished by an external system (e.g. a webhook), the activity can return without completing, and something else completes it later:

```
import { activityInfo, CompleteAsyncError } from '@temporalio/activity';

export async function requestManualReview(docId: string): Promise<string> {
  const token = Buffer.from(activityInfo().taskToken).toString('base64');
  await reviewSystem.createTicket({ docId, callbackToken: token });
  throw new CompleteAsyncError();  // "I'll be completed from outside"
}

// Later, in your webhook handler (client side):
await client.activity.complete(Buffer.from(token, 'base64'), 'APPROVED');
```

6.5 Real-world example: charging a card

The situation. A checkout calls a payment provider. Providers have bad minutes: 503s, timeouts, rate limits. Some failures are final (card declined, invalid amount) and must not be retried; others are transient and must be — without ever charging the customer twice.

Why Temporal fits:

- Retries with exponential back-off are configuration, not code.
- Typed failures separate *declined* (a business result) from *broken* (an error).
- A stable payment ID doubles as the provider's idempotency key, so retries are safe.

```ts
// src/payments/activities.ts
import { ApplicationFailure, activityInfo, log } from '@temporalio/activity';

export interface ChargeRequest {
  paymentId: string; // our ID, also used as the gateway idempotency key  1
  customerId: string;
  amountCents: number;
  currency: string;
}

// The slice of a payment provider (Stripe, Adyen, Razorpay, ...) the activities need
export interface PaymentGateway {
  charge(req: ChargeRequest, idempotencyKey: string): Promise<{ id: string }>;
}
export interface Notifier {
  send(customerId: string, message: string): Promise<void>;
}
```

1 Activities can run more than once; the provider de-duplicates on this key so a retry never charges twice.

```ts
export class GatewayError extends Error {
  constructor(
    message: string,
    readonly status: number,
    readonly declineCode?: string,
  ) {
    super(message);
  }
}

// Dependencies are injected: production passes real clients, tests pass fakes
export function createPaymentActivities(  2
  gateway: PaymentGateway,
  notifier: Notifier,
) {
  return {
    async chargeCard(req: ChargeRequest): Promise<string> {
      if (req.amountCents <= 0) {
        throw ApplicationFailure.nonRetryable(  3
          `Invalid amount: ${req.amountCents}`,
          'InvalidAmount',
        );
      }
      try {
        const charge = await gateway.charge(req, req.paymentId);
        return charge.id;
      } catch (err) {
        if (err instanceof GatewayError && err.status === 402) {  4
          throw ApplicationFailure.create({
            message: `Card declined: ${err.declineCode}`,
            type: 'CardDeclined',
            nonRetryable: true,
            details: [err.declineCode],
          });
        }
        log.warn('Gateway call failed, Temporal will retry', {
          attempt: activityInfo().attempt,
          error: String(err),
        });
        throw err; // 5xx, timeouts: retried  5
      }
    },
```

2 Factory with injected dependencies: real SDK clients in production, fakes in tests. The worker registers the returned object.

3 Bad input can never succeed: fail immediately, no retries.

4 Translate provider errors into Temporal semantics: 402 (declined) is final...

5 ...while anything else is assumed transient and handed to the retry policy.

```
    async sendReceipt(customerId: string, chargeId: string): Promise<void> {
      await notifier.send(customerId, `Payment received (${chargeId})`);
    },

    async notifyDeclined(customerId: string, reason: string): Promise<void> {
      await notifier.send(customerId, `Your payment was declined: ${reason}`);
    },
  };
}

export type PaymentActivities = ReturnType<typeof createPaymentActivities>;  6
```

6 The workflow imports only this **type**, never the implementation.

```
// src/payments/workflows.ts
import {
  proxyActivities,
  ActivityFailure,
  ApplicationFailure,
} from '@temporalio/workflow';
import type { PaymentActivities, ChargeRequest } from './activities';

const { chargeCard } = proxyActivities<PaymentActivities>({  1
  startToCloseTimeout: '10 seconds',
  retry: {
    initialInterval: '1 second', // 1s, 2s, 4s, 8s, 16s between attempts  2
    backoffCoefficient: 2,
    maximumInterval: '30 seconds',
    maximumAttempts: 6,  3
  },
});
const { sendReceipt, notifyDeclined } = proxyActivities<PaymentActivities>({
  startToCloseTimeout: '30 seconds',
});
```

1 Each proxy carries its own policy: the charge is retried, receipts use defaults.

2 Exponential backoff gives a struggling gateway room to recover instead of hammering it.

3 Bounded: after 6 attempts the failure reaches the workflow, which fails visibly in the UI.

```
export type PaymentResult =
  { status: 'PAID'; chargeId: string } | { status: 'DECLINED'; reason: string };

export async function paymentWorkflow(req: ChargeRequest): Promise<PaymentResult> {
  try {
    const chargeId = await chargeCard(req);
    await sendReceipt(req.customerId, chargeId);
    return { status: 'PAID', chargeId };
  } catch (err) {
    if (
      err instanceof ActivityFailure &&
      err.cause instanceof ApplicationFailure &&
      err.cause.type === 'CardDeclined'  4
    ) {
      await notifyDeclined(req.customerId, err.cause.message);
      return { status: 'DECLINED', reason: err.cause.message };  5
    }
    throw err; // InvalidAmount, or retries exhausted: workflow FAILED  6
  }
}
```

4	Branch on the **type** the activity assigned, not on message text.
5	A decline is an expected **business result**: the workflow completes normally with DECLINED.
6	Anything unexpected propagates and fails the execution — visible, alertable, resettable.

```
// test/payments.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { WorkflowFailedError } from '@temporalio/client';
import { ApplicationFailure } from '@temporalio/common';
import {
  createPaymentActivities,
  GatewayError,
  ChargeRequest,
} from '../src/payments/activities';
import { paymentWorkflow } from '../src/payments/workflows';
import { startWorker, uid, RunningWorker } from './helpers';

// Fake payment provider: fails with 503 a few times, declines some customers
class FakeGateway {  1
  calls = 0;
  constructor(
    private transientFailures = 0,
    private declined = new Set<string>(),
  ) {}
  async charge(req: ChargeRequest, key: string) {
    this.calls++;
    if (this.calls <= this.transientFailures) {
      throw new GatewayError('503 Service Unavailable', 503);
    }
    if (this.declined.has(req.customerId)) {
      throw new GatewayError('card_declined', 402, 'insufficient_funds');
    }
    return { id: `ch_${key}` };
  }
}
```

1	A fake provider that misbehaves on demand: 503s, then success; declines for chosen customers.

```
let env: TestWorkflowEnvironment;
let worker: RunningWorker;
let gateway = new FakeGateway();
const messages: string[] = [];

before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();  2
  const activities = createPaymentActivities(
    { charge: (req, key) => gateway.charge(req, key) }, // swap per test
    {
      send: async (_c, msg) => {
        messages.push(msg);
      },
    },
  );
  worker = await startWorker(
    env,
    'payments',
    require.resolve('../src/payments/workflows'),
    activities,
  );
});
after(async () => {
  await worker.stop();
  await env.teardown();
});
```

2 Real Temporal semantics in-process; retry back-off delays are skipped.

```
const pay = (over: Partial<ChargeRequest> = {}) =>
  env.client.workflow.execute(paymentWorkflow, {
    taskQueue: 'payments',
    workflowId: uid('pay'),
    args: [
      {
        paymentId: uid('p'),
        customerId: 'cus_1',
        amountCents: 4999,
        currency: 'USD',
        ...over,
      },
    ],
  });
```

```
test('transient gateway errors are retried until the charge succeeds', async () => {
  gateway = new FakeGateway(2); // two 503s, then success
  const result = await pay();
  assert.equal(result.status, 'PAID');
  assert.equal(gateway.calls, 3);  3
  assert.match(messages.at(-1)!, /Payment received/);
});

test('a declined card is a business outcome, not a failure', async () => {
  gateway = new FakeGateway(0, new Set(['cus_broke']));
  const result = await pay({ customerId: 'cus_broke' });
  assert.deepEqual(result, {
    status: 'DECLINED',
    reason: 'Card declined: insufficient_funds',
  });
  assert.equal(gateway.calls, 1); // non-retryable: no retries  4
});
```

3 Proof of the retry policy: two 503s, success on the third call.

4 Proof that non-retryable errors are not retried.

```
test('an invalid amount fails the workflow without calling the gateway', async () => {
  gateway = new FakeGateway();
  await assert.rejects(pay({ amountCents: 0 }), (err: unknown) => {
    assert.ok(err instanceof WorkflowFailedError);
    const cause = (err.cause as Error).cause;
    assert.ok(cause instanceof ApplicationFailure);
    assert.equal(cause.type, 'InvalidAmount');
    return true;
  });
  assert.equal(gateway.calls, 0);
});
```

What the tests prove (`test/payments.test.ts`, all passing):

Test	Verified behaviour
transient gateway errors are retried until the charge succeeds	Two 503s, then success: exactly 3 gateway calls, receipt sent, result PAID.
a declined card is a business outcome, not a failure	Workflow **completes** with DECLINED; 1 call only (non-retryable).
an invalid amount fails the workflow without calling the gateway	Workflow FAILED; root cause `InvalidAmount`; 0 gateway calls.

7. Workflows

7.1 Anatomy of a workflow

```
// src/workflows.ts
import { proxyActivities, log, workflowInfo, sleep } from '@temporalio/workflow';
import type * as activities from './activities';

const { sendEmail, getExchangeRate } = proxyActivities<typeof activities>({
  startToCloseTimeout: '30 seconds',
});

export interface InvoiceInput {
  customerEmail: string; amountUsd: number; currency: string;
}

export async function invoiceWorkflow(input: InvoiceInput): Promise<number> {
  const { workflowId } = workflowInfo();
  log.info('Invoice started', { workflowId });           // replay-safe logger

  const rate = await getExchangeRate('USD', input.currency);
  const amount = Math.round(input.amountUsd * rate * 100) / 100;

  await sendEmail(input.customerEmail, `Invoice: ${amount} ${input.currency}`);
  await sleep('7 days');                                  // durable timer
  await sendEmail(input.customerEmail, 'Payment reminder');
  return amount;
}
```

Tip: Take a **single object argument**. You can add optional fields later without breaking running workflows or callers.

7.2 Running steps in parallel

```
// Parallel activities - plain Promise.all works and is deterministic
const [user, cart, prefs] = await Promise.all([
  getUser(userId), getCart(userId), getPreferences(userId),
]);

// Race: first to finish wins
const winner = await Promise.race([fastProvider(q), slowProvider(q)]);
```

7.3 Local activities

`proxyLocalActivities` runs short activities (< a few seconds) inside the same workflow task, without a round-trip through the task queue. Lower latency and fewer history events; use for quick lookups or validations.

```
import { proxyLocalActivities } from '@temporalio/workflow';
const { validateAddress } = proxyLocalActivities<typeof activities>({
  startToCloseTimeout: '5 seconds',
});
```

7.4 Helpers available inside workflows

API	Purpose
`workflowInfo()`	workflowId, runId, workflowType, taskQueue, attempt, continueAsNewSuggested, historyLength, ...
`log`	Replay-aware logger (does not re-log during replay).
`uuid4()`	Deterministic UUID (use instead of crypto.randomUUID).
`sleep(duration)`	Durable timer. Accepts ms or strings like '5 minutes'.
`condition(fn, timeout?)`	Wait until fn() is true; returns false on timeout.
`upsertSearchAttributes()`	Update indexed fields visible to list queries.
`patched(id)`	Versioning branch for code changes.

7.5 Real-world example: flight search across providers

The situation. A travel site queries three airline/aggregator APIs for every search and shows the cheapest offer. Each API takes ~500 ms; one is down today.

Why Temporal fits:

- Parallel activities with `Promise.allSettled` — fan-out and fan-in in plain TypeScript.
- A local activity validates the query without a task-queue round trip.
- One failing provider degrades the result instead of breaking the search.

```ts
// src/travel-search/workflows.ts
import {
  proxyActivities,
  proxyLocalActivities,
  ApplicationFailure,
} from '@temporalio/workflow';
import type { SearchActivities, SearchQuery, Offer } from './activities';

const { validateQuery } = proxyLocalActivities<SearchActivities>({  1
  startToCloseTimeout: '2 seconds',
});
const { searchProvider } = proxyActivities<SearchActivities>({
  startToCloseTimeout: '5 seconds',
  retry: { maximumAttempts: 2 }, // a slow provider must not stall the search  2
});
```

1 **Local activity**: runs inside the workflow task on the same worker — ideal for quick, pure checks.

2 Search results go stale quickly: fail fast rather than retry for minutes.

```ts
export interface SearchResult {
  cheapest: Offer;
  offersFound: number;
  failedProviders: string[];
}

export async function flightSearchWorkflow(input: {
  query: SearchQuery;
  providers: string[];
}): Promise<SearchResult> {
  const query = await validateQuery(input.query);

  // Fan out: every provider is searched in parallel
  const settled = await Promise.allSettled(  3
    input.providers.map((name) => searchProvider(name, query)),
  );
```

3 Fan-out: one activity per provider, all in flight at once. `Promise.allSettled` is deterministic inside workflows.

```ts
  // Fan in: keep what succeeded, remember what failed
  const offers: Offer[] = [];
  const failedProviders: string[] = [];
  settled.forEach((r, i) => {
    if (r.status === 'fulfilled') offers.push(...r.value);
    else failedProviders.push(input.providers[i]);  4
  });

  if (offers.length === 0) {
    throw ApplicationFailure.nonRetryable(  5
      'No provider returned offers',
      'NoOffers',
    );
  }
  offers.sort((a, b) => a.priceCents - b.priceCents);
  return { cheapest: offers[0], offersFound: offers.length, failedProviders };
}
```

4 Partial failure is normal for aggregators: keep the offers that arrived.

5 Only when every provider failed is the search itself a failure.

What the tests prove (`test/travel-search.test.ts`, all passing):

Test	Verified behaviour
searches providers in parallel and returns the cheapest offer	Three 500 ms providers finish in < 1.5 s (concurrent); cheapest from `jetnow`; `budgetfly` reported as failed.
an invalid query is rejected by the local activity	Same origin and destination: workflow FAILED with the validation message.

8. Workers

8.1 A production-ready worker

```
// src/worker.ts
import { NativeConnection, Worker, Runtime, DefaultLogger } from '@temporalio/worker';
import * as activities from './activities';
import { TASK_QUEUE } from './shared';

Runtime.install({ logger: new DefaultLogger('INFO') });   // optional, once per process

async function run() {
  const connection = await NativeConnection.connect({
    address: process.env.TEMPORAL_ADDRESS ?? 'localhost:7233',
  });
  try {
    const worker = await Worker.create({
      connection,
      namespace: process.env.TEMPORAL_NAMESPACE ?? 'default',
      taskQueue: TASK_QUEUE,
      workflowsPath: require.resolve('./workflows'),
      activities,
      maxConcurrentActivityTaskExecutions: 100,
      maxConcurrentWorkflowTaskExecutions: 40,
      shutdownGraceTime: '30 seconds',   // let activities finish on SIGTERM
    });
    console.log(`Worker polling task queue "${TASK_QUEUE}"`);
    await worker.run();                  // resolves after graceful shutdown
  } finally {
    await connection.close();
  }
}

run().catch((err) => { console.error(err); process.exit(1); });
```

8.2 Worker lifecycle

1. **Create**: connects, bundles workflow code (a few seconds; see pre-bundling in Chapter 20), registers activities.
2. **Run**: long-polls the task queue for workflow and activity tasks, executes them, keeps a cache of running workflows.
3. **Shutdown**: on SIGINT/SIGTERM (default) or `worker.shutdown()` it stops polling, waits up to `shutdownGraceTime` for in-flight activities, then `run()` resolves.

8.3 Things to know

- Workers are **stateless** from the server's view. Run as many replicas as you need; tasks are load-balanced automatically.
- All workers on one task queue must register the **same** workflows and activities.
- You can split workflows and activities into different workers/queues (e.g. a GPU worker for a `transcode` queue). Set `taskQueue` in `proxyActivities` options to route activities.

- If no worker is running, nothing fails — tasks wait in the queue until one starts.

9. Clients

9.1 Connecting

```
import { Connection, Client } from '@temporalio/client';

const connection = await Connection.connect({ address: 'localhost:7233' });
const client = new Client({ connection, namespace: 'default' });
// Create ONE client per process and reuse it (e.g. export from a module).
```

9.2 Starting workflows

```
import { invoiceWorkflow } from './workflows';
import { defineSearchAttributeKey, SearchAttributeType } from '@temporalio/common';

// Register first (CLI):
//   temporal operator search-attribute create --name CustomerId --type Keyword
const CustomerId = defineSearchAttributeKey('CustomerId', SearchAttributeType.KEYWORD);

// start: returns immediately with a handle
const handle = await client.workflow.start(invoiceWorkflow, {
  taskQueue: 'orders',
  workflowId: `invoice-${invoiceId}`,          // business ID = natural dedup
  args: [{ customerEmail: 'a@b.com', amountUsd: 120, currency: 'EUR' }],
  workflowExecutionTimeout: '30 days',          // optional hard cap
  memo: { createdBy: 'billing-api' },           // unindexed metadata
  typedSearchAttributes: [{ key: CustomerId, value: 'c-42' }],   // indexed
});

// execute: start + wait for the result
const amount = await client.workflow.execute(invoiceWorkflow, {
  taskQueue: 'orders', workflowId: `invoice-${invoiceId}`, args: [input],
});
```

9.3 Working with existing executions

```
const handle = client.workflow.getHandle('invoice-123');   // no network call yet

const result = await handle.result();          // waits for completion
const desc   = await handle.describe();        // status, start time, memo, ...
console.log(desc.status.name);                 // RUNNING | COMPLETED | FAILED | ...

await handle.cancel();                         // graceful: raises CancelledFailure inside
await handle.terminate('bad data');            // immediate hard stop, no cleanup

// List with a visibility query
for await (const wf of client.workflow.list({
  query: `WorkflowType = 'invoiceWorkflow' AND ExecutionStatus = 'Running'`,
})) {
  console.log(wf.workflowId, wf.startTime);
}
```

9.4 Workflow ID policies

Option	Values and effect
`workflowIdReusePolicy`	What happens when a **closed** run with the same ID exists: ALLOW_DUPLICATE (default), ALLOW_DUPLICATE_FAILED_ONLY, REJECT_DUPLICATE.
`workflowIdConflictPolicy`	What happens when a **running** run with the same ID exists: FAIL (default, throws WorkflowExecutionAlreadyStartedError), USE_EXISTING, TERMINATE_EXISTING.

10. First Run: Hello World End to End

Put everything together and watch the runtime flow from Chapter 2 happen for real.

Four small files (worker.ts is the one from Chapter 8)

```
// src/shared.ts
export const TASK_QUEUE = 'hello-tq';

// src/activities.ts
export async function greet(name: string): Promise<string> {
  return `Hello, ${name}!`;
}

// src/workflows.ts
import { proxyActivities } from '@temporalio/workflow';
import type * as activities from './activities';
const { greet } = proxyActivities<typeof activities>({ startToCloseTimeout: '1 minute' });

export async function greetingWorkflow(name: string): Promise<string> {
  return await greet(name);
}

// src/client.ts
import { Connection, Client } from '@temporalio/client';
import { greetingWorkflow } from './workflows';
import { TASK_QUEUE } from './shared';

async function run() {
  const client = new Client({ connection: await Connection.connect() });
  const handle = await client.workflow.start(greetingWorkflow, {
    taskQueue: TASK_QUEUE, args: ['Temporal'], workflowId: 'hello-' + Date.now(),
  });
  console.log('Started', handle.workflowId);
  console.log(await handle.result());
}
run().catch((e) => { console.error(e); process.exit(1); });
```

Run

```
temporal server start-dev      # terminal 1
npm run worker                 # terminal 2
npm run client                 # terminal 3
# -> Started hello-1727512345678
# -> Hello, Temporal!
```

10.1 Reading the event history

Open `http://localhost:8233`, click the workflow, and look at the history (or run `temporal workflow show -w hello-...`):

#	Event	Meaning
1	WorkflowExecutionStarted	Client call recorded with args and task queue.
2-4	WorkflowTask Scheduled/Started/Completed	Worker ran code until `await greet()`; returned a ScheduleActivity command.
5	ActivityTaskScheduled	Activity task placed on the queue.
6	ActivityTaskStarted	Worker picked it up (written with the completion).
7	ActivityTaskCompleted	Result "Hello, Temporal!" stored.
8-10	WorkflowTask Scheduled/Started/Completed	Worker resumed the workflow with the result; it returned.
11	WorkflowExecutionCompleted	Final result stored.

10.2 Prove durability

1. Change the workflow to `await sleep('1 minute')` between two activity calls.
2. Start it, then kill the worker (Ctrl+C) while it sleeps.
3. Wait past the minute, then restart the worker.
4. The workflow continues immediately after the sleep; the first activity is **not** executed again. That is replay in action.

Part III — Workflow Features

11. Signals, Queries & Updates

Message	Direction	Can change state?	Returns value?	Needs running worker?
Signal	into workflow	Yes	No (fire & forget)	No (buffered)
Query	read from workflow	No (read-only)	Yes	Yes
Update	into workflow	Yes	Yes (waits for handler)	Yes

11.1 Real-world example: a shopping cart

The situation. Each shopper's cart is a workflow (ID `cart-{userId}`). The storefront adds and removes items, shows the cart, applies coupons and checks out. Abandoned carts get one reminder after 24 hours and expire after 48.

Why Temporal fits:

- Signals for fire-and-forget changes, a query for the cart page, updates for operations that need an answer (coupon %, order ID).
- Update validators reject bad requests before they touch history.
- Idle detection with `condition` + timeout: no cron, no sweeper job.

```
// src/shopping-cart/workflows.ts
import {
  proxyActivities,
  defineSignal,
  defineQuery,
  defineUpdate,
  setHandler,
  condition,
  allHandlersFinished,
} from '@temporalio/workflow';
import type { Duration } from '@temporalio/common';
import type { CartActivities } from './activities';

const { createOrder, sendCartReminder } = proxyActivities<CartActivities>({
  startToCloseTimeout: '30 seconds',
});
```

```
export interface CartItem {
  sku: string;
  qty: number;
  priceCents: number;
}
export interface CartView {
  items: CartItem[];
  discountPct: number;
  totalCents: number;
  orderId?: string;
}
export type CartOutcome =
  | { status: 'ORDERED'; orderId: string; totalCents: number }
  | { status: 'EXPIRED'; itemsLeft: number };
```

```ts
// Signals: fire-and-forget changes. Query: read-only view.
// Updates: validated changes that send a reply.
export const addItem = defineSignal<[CartItem]>('addItem');  1
export const removeItem = defineSignal<[string]>('removeItem');
export const getCart = defineQuery<CartView>('getCart');  2
export const applyCoupon = defineUpdate<number, [string]>('applyCoupon');  3
export const checkout = defineUpdate<string, []>('checkout');

const COUPONS: Record<string, number> = { SAVE10: 10, WELCOME20: 20 };

export async function cartWorkflow({
  userId,
  idleTimeout = '24 hours',
}: {
  userId: string;
  idleTimeout?: Duration;
}): Promise<CartOutcome> {
  const items = new Map<string, CartItem>();
  let discountPct = 0;
  let orderId: string | undefined;
  let checkingOut = false;
  let version = 0; // bumps on every change: used for idle detection  4
  let reminded = false;
```

1	**Signal**: the storefront fires it and moves on; delivered even if no worker is running.
2	**Query**: read-only snapshot for the cart page; never changes history.
3	**Update**: a change that returns a value to the caller (the discount).
4	Any change resets the idle clock below.

```ts
  const totalCents = () => {
    const gross = [...items.values()].reduce((s, i) => s + i.qty * i.priceCents, 0);
    return Math.round((gross * (100 - discountPct)) / 100);
  };

  setHandler(addItem, (item) => {
    const qty = (items.get(item.sku)?.qty ?? 0) + item.qty;
    items.set(item.sku, { ...item, qty });
    version++;
  });
  setHandler(removeItem, (sku) => {
    items.delete(sku);
    version++;
  });
  setHandler(getCart, () => ({
    items: [...items.values()],
    discountPct,
    totalCents: totalCents(),
    orderId,
  }));
```

```
  setHandler(
    applyCoupon,
    (code) => {
      discountPct = COUPONS[code];
      version++;
      return discountPct;
    },
    {
      validator: (code) => {  5
        // runs before anything is recorded
        if (!(code in COUPONS)) throw new Error(`Unknown coupon ${code}`);
      },
    },
  );
```

5 Validators run **before** the update is recorded: an unknown coupon is rejected without touching history.

```
  setHandler(
    checkout,
    async () => {
      checkingOut = true;
      orderId = await createOrder(userId, [...items.values()], totalCents());  6
      return orderId;
    },
    {
      validator: () => {
        if (items.size === 0) throw new Error('Cart is empty');
        if (checkingOut) throw new Error('Checkout already in progress');
      },
    },
  );
```

6 Update handlers may be async and call activities; the reply is sent when the order exists.

```
  // Wait for activity; after one idle period send a reminder, after two expire
  while (orderId === undefined) {
    const seen = version;
    const active = await condition(  7
      () => orderId !== undefined || checkingOut || version !== seen,
      idleTimeout,
    );
    if (active) {
      if (checkingOut) await condition(() => orderId !== undefined);
      continue;
    }
    if (reminded || items.size === 0)
      return { status: 'EXPIRED', itemsLeft: items.size };
    await sendCartReminder(userId, items.size);  8
    reminded = true;
  }

  await condition(allHandlersFinished); // let the checkout update reply first  9
  return { status: 'ORDERED', orderId, totalCents: totalCents() };
}
```

7 Wait for a change *or* the idle timeout, whichever comes first.

8 After 24 idle hours: one reminder e-mail. After another 24: EXPIRED.

9 Never return while an update handler is still running.

```ts
// test/shopping-cart.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { WorkflowUpdateFailedError } from '@temporalio/client';
import { createCartActivities } from '../src/shopping-cart/activities';
import {
  cartWorkflow,
  addItem,
  removeItem,
  getCart,
  applyCoupon,
  checkout,
} from '../src/shopping-cart/workflows';
import { startWorker, uid, RunningWorker } from './helpers';
```

```ts
let env: TestWorkflowEnvironment;
let worker: RunningWorker;
const mails: string[] = [];
let orders = 0;

before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();
  const activities = createCartActivities(
    { create: async () => `ord_${++orders}` },
    {
      send: async (userId, template) => {
        mails.push(`${userId}:${template}`);
      },
    },
  );
  worker = await startWorker(
    env,
    'cart',
    require.resolve('../src/shopping-cart/workflows'),
    activities,
  );
});
after(async () => {
  await worker.stop();
  await env.teardown();
});
```

```ts
test('signals change the cart, queries read it, updates reply', async () => {
  const handle = await env.client.workflow.start(cartWorkflow, {
    taskQueue: 'cart',
    workflowId: uid('cart'),
    args: [{ userId: 'u1' }],
  });
  await handle.signal(addItem, { sku: 'BOOK', qty: 1, priceCents: 1500 });  // 1
  await handle.signal(addItem, { sku: 'BOOK', qty: 1, priceCents: 1500 });
  await handle.signal(addItem, { sku: 'PEN', qty: 3, priceCents: 200 });
  await handle.signal(removeItem, 'PEN');
  assert.equal((await handle.query(getCart)).totalCents, 3000);  // 2

  await assert.rejects(
    handle.executeUpdate(applyCoupon, { args: ['FREE100'] }),  // 3
    WorkflowUpdateFailedError,
  ); // rejected by validator
  assert.equal(await handle.executeUpdate(applyCoupon, { args: ['SAVE10'] }), 10);
```

1. `handle.signal()` returns as soon as the server has stored the signal.
2. Two books (merged), the pens removed: the query sees the current state.
3. Validator rejection surfaces as `WorkflowUpdateFailedError`.

```
  const orderId = await handle.executeUpdate(checkout); // reply = new order ID  4
  assert.match(orderId, /^ord_/);
  assert.deepEqual(await handle.result(), {
    status: 'ORDERED',
    orderId,
    totalCents: 2700,
  });
});

test('an abandoned cart gets one reminder, then expires', async () => {
  const handle = await env.client.workflow.start(cartWorkflow, {
    taskQueue: 'cart',
    workflowId: uid('cart'),
    args: [{ userId: 'u2' }],
  });
  await handle.signal(addItem, { sku: 'SHOES', qty: 1, priceCents: 8900 });
  const result = await handle.result(); // 48 h skipped  5
  assert.deepEqual(result, { status: 'EXPIRED', itemsLeft: 1 });
  assert.deepEqual(
    mails.filter((m) => m.startsWith('u2')),
    ['u2:abandoned-cart'],
  );
});
```

4	The caller gets the new order ID synchronously — no polling.
5	Two 24-hour idle timers elapse in milliseconds under time skipping.

```
test('checkout of an empty cart is rejected', async () => {
  const handle = await env.client.workflow.start(cartWorkflow, {
    taskQueue: 'cart',
    workflowId: uid('cart'),
    args: [{ userId: 'u3' }],
  });
  await assert.rejects(handle.executeUpdate(checkout), WorkflowUpdateFailedError);
  await handle.terminate('test done');
});
```

What the tests prove (`test/shopping-cart.test.ts`, all passing):

Test	Verified behaviour
signals change the cart, queries read it, updates reply	Merged quantities, removal, invalid coupon rejected, SAVE10 = 10 %, checkout returns the order ID, total 2700.
an abandoned cart gets one reminder, then expires	Exactly one reminder e-mail, result EXPIRED (48 h skipped).
checkout of an empty cart is rejected	Validator rejects; nothing recorded.

12. Timers & Schedules

12.1 Durable timers

```
import { sleep, condition } from '@temporalio/workflow';

await sleep('30 days');                  // survives restarts; zero cost while waiting
await sleep(5000);                       // milliseconds also accepted

// Wait for a signal OR a timeout
const approved = await condition(() => approvedBy !== undefined, '48 hours');
if (!approved) { /* escalate */ }

// Timer vs activity race
const result = await Promise.race([
  slowActivity(),
  sleep('10 minutes').then(() => 'TIMED_OUT' as const),
]);
```

12.2 Schedules (replacement for cron)

```
import { ScheduleOverlapPolicy } from '@temporalio/client';

await client.schedule.create({
  scheduleId: 'daily-sales-report',
  spec: {
    calendars: [{ hour: 2, minute: 0 }],     // or cronExpressions: ['0 2 * * *']
    // intervals: [{ every: '15 minutes' }],
  },
  action: {
    type: 'startWorkflow',
    workflowType: reportWorkflow,
    taskQueue: 'reports',
    args: [{ region: 'EU' }],
  },
  policies: { overlap: ScheduleOverlapPolicy.SKIP, catchupWindow: '1 hour' },
});

const sched = client.schedule.getHandle('daily-sales-report');
await sched.trigger();                         // run now
await sched.pause('maintenance window');
await sched.unpause();
await sched.delete();
```

CLI equivalents

```
temporal schedule list
temporal schedule describe --schedule-id daily-sales-report
temporal schedule trigger  --schedule-id daily-sales-report
```

12.3 Real-world example: a 14-day free trial

The situation. Every SaaS sign-up starts a trial: welcome e-mail, reminder on day 11, downgrade on day 14 — unless the customer upgrades first.

Why Temporal fits:

- One workflow per sign-up holds the whole lifecycle; no nightly batch job.
- Timers survive deploys and outages and cost nothing while waiting.
- Tests run 14 days in seconds.

```ts
// src/free-trial/workflows.ts
import {
  proxyActivities,
  defineSignal,
  defineQuery,
  setHandler,
  condition,
} from '@temporalio/workflow';
import type { TrialActivities } from './activities';

const { sendEmail, activatePlan, downgradeToFree } =
  proxyActivities<TrialActivities>({
    startToCloseTimeout: '1 minute',
  });
```

```ts
const DAY = 24 * 60 * 60 * 1000;

export const upgraded = defineSignal<[string]>('upgraded'); // from the billing page  1
export const trialStatus = defineQuery<string>('trialStatus');

export interface TrialInput {
  userId: string;
  email: string;
  trialDays?: number;
}

// One workflow per sign-up: runs for 14 days, costs nothing while it waits  2
export async function freeTrialWorkflow({
  userId,
  email,
  trialDays = 14,
}: TrialInput): Promise<'CONVERTED' | 'EXPIRED'> {
  let plan: string | undefined;
  let status = 'TRIAL_ACTIVE';
  setHandler(upgraded, (p) => {
    plan = p;
  });
  setHandler(trialStatus, () => status);
```

1 The billing page signals the workflow when the customer pays.

2 No cron, no "trials ending today" SQL scan: each trial is its own durable timeline.

```ts
  const convert = async () => {
    status = `CONVERTED:${plan}`;
    await activatePlan(userId, plan!);
    await sendEmail(email, 'thanks-for-upgrading');
    return 'CONVERTED' as const;
  };

  await sendEmail(email, 'welcome');

  // Durable timer + early exit: wake up after 11 days OR as soon as they upgrade
  if (await condition(() => plan !== undefined, (trialDays - 3) * DAY))  3
    return convert();

  status = 'ENDING_SOON';
  await sendEmail(email, 'trial-ends-in-3-days');  4
  if (await condition(() => plan !== undefined, 3 * DAY)) return convert();
```

3 `condition` with a timeout = durable timer with an early exit. Survives deploys and restarts for the full 11 days.

4 Day 11: the reminder.

```
  status = 'EXPIRED';
  await downgradeToFree(userId);  5
  await sendEmail(email, 'trial-expired');
  return 'EXPIRED';
}
```

5 Day 14 without payment: downgrade, then tell the customer.

```
// test/free-trial.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { createTrialActivities } from '../src/free-trial/activities';
import {
  freeTrialWorkflow,
  upgraded,
  trialStatus,
} from '../src/free-trial/workflows';
import { startWorker, uid, RunningWorker } from './helpers';

let env: TestWorkflowEnvironment;
let worker: RunningWorker;
const log: string[] = [];
```

```
before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();
  const activities = createTrialActivities(
    {
      activatePlan: async (u, plan) => {
        log.push(`${u}:activate:${plan}`);
      },
      downgradeToFree: async (u) => {
        log.push(`${u}:downgrade`);
      },
    },
    {
      send: async (email, template) => {
        log.push(`${email}:${template}`);
      },
    },
  );
  worker = await startWorker(
    env,
    'trial',
    require.resolve('../src/free-trial/workflows'),
    activities,
  );
});
after(async () => {
  await worker.stop();
  await env.teardown();
});
```

```
test('14 silent days: reminder on day 11, downgrade on day 14', async () => {
  const started = Date.now();
  const result = await env.client.workflow.execute(freeTrialWorkflow, {
    taskQueue: 'trial',
    workflowId: uid('trial'),
    args: [{ userId: 'u1', email: 'ann@x.com' }],
  });
  assert.equal(result, 'EXPIRED');
  assert.deepEqual(
    log.filter((l) => l.startsWith('ann') || l.startsWith('u1')),
    [
      'ann@x.com:welcome',
      'ann@x.com:trial-ends-in-3-days',
      'u1:downgrade',
      'ann@x.com:trial-expired',
    ],
  );
  assert.ok(Date.now() - started < 10_000, '14 days of timers are skipped');  1
});
```

1 14 days of timers complete in seconds.

```
test('upgrading on day 5 converts immediately, no reminder', async () => {
  const handle = await env.client.workflow.start(freeTrialWorkflow, {
    taskQueue: 'trial',
    workflowId: uid('trial'),
    args: [{ userId: 'u2', email: 'bob@x.com' }],
  });
  await env.sleep('5 days'); // advance the test clock  2
  assert.equal(await handle.query(trialStatus), 'TRIAL_ACTIVE');
  await handle.signal(upgraded, 'pro');  3
  assert.equal(await handle.result(), 'CONVERTED');
  assert.deepEqual(
    log.filter((l) => l.startsWith('bob') || l.startsWith('u2')),
    ['bob@x.com:welcome', 'u2:activate:pro', 'bob@x.com:thanks-for-upgrading'],
  );
});
```

2 `env.sleep` moves virtual time forward deterministically.

3 The upgrade wakes the waiting `condition` immediately.

What the tests prove (`test/free-trial.test.ts`, all passing):

Test	Verified behaviour
14 silent days: reminder on day 11, downgrade on day 14	E-mails and downgrade in the right order; whole run < 10 s.
upgrading on day 5 converts immediately, no reminder	Query shows TRIAL_ACTIVE on day 5; signal converts; no downgrade.

12.4 Real-world example: the nightly sales report

The situation. Finance wants yesterday's sales per region in their inbox at 02:30 India time, every day, even if the server was briefly down at that moment.

Why Temporal fits:

- Schedules replace cron: time zones, overlap and catch-up policies, pause/trigger from code, CLI or UI.
- Each run is an ordinary workflow with retries and history.

```ts
// src/reports/workflows.ts
import { proxyActivities } from '@temporalio/workflow';
import type { ReportActivities, SalesSummary } from './activities';

const { aggregateSales, emailReport } = proxyActivities<ReportActivities>({
  startToCloseTimeout: '10 minutes',
  retry: { initialInterval: '30 seconds', maximumAttempts: 10 },
});

// Started by a Schedule every night; reports on the previous calendar day
export async function dailySalesReportWorkflow({
  region,
  recipients,
}: {
  region: string;
  recipients: string[];
}): Promise<SalesSummary> {
  // Date.now() is deterministic in workflows: the time of the current workflow task
  const day = new Date(Date.now() - 24 * 60 * 60 * 1000).toISOString().slice(0, 10);  1
  const summary = await aggregateSales(region, day);
  await Promise.all(recipients.map((to) => emailReport(to, summary)));  2
  return summary;
}
```

1	Safe in a workflow: the sandbox makes `Date.now()` replay-stable.
2	One activity per recipient: a bounced address doesn't block the others.

```ts
// src/reports/schedule.ts
import { Client, ScheduleOverlapPolicy } from '@temporalio/client';
import { dailySalesReportWorkflow } from './workflows';

// Run once at deploy time (idempotent: delete + recreate, or catch "already exists")
export async function createNightlySalesSchedule(client: Client, region: string) {
  return client.schedule.create({
    scheduleId: `nightly-sales-${region}`,
    spec: {
      calendars: [{ hour: 2, minute: 30 }], // 02:30 every day...  1
      timezone: 'Asia/Kolkata', // ...in the business's time zone  2
    },
    action: {
      type: 'startWorkflow',
      workflowType: dailySalesReportWorkflow,
      taskQueue: 'reports',
      args: [
        { region, recipients: ['cfo@corp.example', 'sales-leads@corp.example'] },
      ],
    },
    policies: {
      overlap: ScheduleOverlapPolicy.SKIP, // yesterday's run still going? skip  3
      catchupWindow: '6 hours', // server was down at 02:30? run late  4
    },
  });
}
```

1	Calendar spec (cron expressions also work).
2	Time zones and daylight-saving changes are handled by the server.
3	Overlap policy: SKIP, BUFFER_ONE, ALLOW_ALL, CANCEL_OTHER, TERMINATE_OTHER...
4	Missed runs within the window are executed once the server is back.

```ts
// test/reports.test.ts
// Schedules need a full Temporal server: this test uses the local dev server
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { setTimeout as delay } from 'node:timers/promises';
import { TestWorkflowEnvironment } from '@temporalio/testing';
```

```
import { createReportActivities } from '../src/reports/activities';
import { createNightlySalesSchedule } from '../src/reports/schedule';
import { startWorker, RunningWorker } from './helpers';

let env: TestWorkflowEnvironment;
let worker: RunningWorker;
const mails: string[] = [];

before(async () => {
  env = await TestWorkflowEnvironment.createLocal();  1
  const activities = createReportActivities(
    {
      salesFor: async (region, day) => ({
        region,
        day,
        orders: 1284,
        revenueCents: 9_876_500,
      }),
    },
    {
      send: async (to, subject) => {
        mails.push(`${to}|${subject}`);
      },
    },
  );
  worker = await startWorker(
    env,
    'reports',
    require.resolve('../src/reports/workflows'),
    activities,
  );
});
after(async () => {
  await worker.stop();
  await env.teardown();
});
```

1 Schedules need a full server: this test runs on the real dev server.

```
test('nightly schedule: next run 02:30 IST, trigger, pause, delete', async () => {
  const schedule = await createNightlySalesSchedule(env.client, 'south');
  const before = await schedule.describe();
  const next = before.info.nextActionTimes[0];
  // 02:30 in Asia/Kolkata (UTC+5:30) is 21:00 UTC
  assert.equal(next.toISOString().slice(11, 16), '21:00');  2

  await schedule.trigger(); // run it right now  3
  for (let i = 0; i < 100 && mails.length < 2; i++) await delay(100);
  assert.equal(mails.length, 2);
  assert.match(mails[0], /\|Sales south \d{4}-\d{2}-\d{2}$/);

  await schedule.pause('Quarter-end freeze');  4
  assert.equal((await schedule.describe()).state.paused, true);
  await schedule.delete();
});
```

2 02:30 IST is 21:00 UTC — verified from the schedule's next action time.

3 Run on demand (e.g. a manager wants today's numbers now).

4 Pause with a note; visible in the UI and CLI.

What the tests prove (`test/reports.test.ts`, all passing):

Test	Verified behaviour
nightly schedule: next run 02:30 IST, trigger, pause, delete	Next action time is 21:00 UTC; a manual trigger e-mails both recipients; pause sets `state.paused`. Runs on the real dev server.

13. Error Handling, Retries & Timeouts

13.1 Activity timeouts

Timeout	What it limits	Guidance
`startToCloseTimeout`	One attempt of the activity.	**Always set** (or scheduleToClose). Slightly above worst-case run time.
`scheduleToCloseTimeout`	Whole activity incl. all retries.	Use to bound total effort, e.g. '10 minutes'.
`scheduleToStartTimeout`	Time waiting in the queue.	Rarely needed; for routing to a fallback queue.
`heartbeatTimeout`	Max gap between heartbeats.	Set for long activities so crashes are detected fast.

13.2 Retry policy

```
const { chargeCard } = proxyActivities<typeof activities>({
  startToCloseTimeout: '20 seconds',
  retry: {
    initialInterval: '1 second',     // first retry delay
    backoffCoefficient: 2,           // 1s, 2s, 4s, 8s ...
    maximumInterval: '1 minute',     // cap per-retry delay
    maximumAttempts: 10,             // 0 or unset = unlimited
    nonRetryableErrorTypes: ['CardDeclined', 'InvalidAmount'],
  },
});
```

13.3 Throwing errors from activities

```
import { ApplicationFailure } from '@temporalio/activity';

export async function chargeCard(orderId: string, cents: number): Promise<string> {
  if (cents <= 0) {
    throw ApplicationFailure.nonRetryable('Amount must be positive', 'InvalidAmount');
  }
  const r = await gateway.charge({ idempotencyKey: orderId, cents });
  if (r.status === 'declined') {
    throw ApplicationFailure.create({
      message: 'Card declined', type: 'CardDeclined', nonRetryable: true,
      details: [{ reason: r.reason }],
    });
  }
  if (r.status === 'rate_limited') {
    throw ApplicationFailure.create({         // retryable, custom delay
      message: 'Rate limited', nextRetryDelay: '30 seconds',
    });
  }
  return r.transactionId;          // network errors etc. just throw -> retried
}
```

13.4 Catching errors in workflows

```
import { ActivityFailure, ApplicationFailure } from '@temporalio/workflow';

try {
  await chargeCard(orderId, cents);
} catch (err) {
  if (err instanceof ActivityFailure && err.cause instanceof ApplicationFailure) {
    if (err.cause.type === 'CardDeclined') {
      await sendEmail(email, 'Your card was declined');
      return 'DECLINED';
    }
  }
  throw err;   // anything else: fail the workflow
}
```

Failure class	When you see it
`ActivityFailure`	Wrapper when an activity finally fails; `.cause` holds the reason.
`ApplicationFailure`	Your own business error (from activity or workflow).
`TimeoutFailure`	An activity/child timed out (`timeoutType` says which).
`CancelledFailure`	Scope or workflow was cancelled.
`ChildWorkflowFailure`	A child workflow failed; `.cause` has details.
`WorkflowFailedError`	(Client side) thrown by `handle.result()` when the workflow failed.

Watch out: In a **workflow**, throwing a plain `Error` does NOT fail the execution — it fails the workflow *task*, which is retried forever (so you can deploy a fix). To deliberately fail the workflow, throw `ApplicationFailure`.

13.5 Real-world example: a loan application

The situation. An application is validated, scored by a credit bureau (2-second SLA), fraud-checked and decided. The bureau sometimes hangs.

Why Temporal fits:

- Three kinds of outcome, three mechanisms: **invalid input** fails the workflow, **business rules** return a decision, **infrastructure trouble** is caught and routed to a human.
- Timeouts are typed (`TimeoutFailure`) and can be handled like any other error.

```ts
// src/loan-application/workflows.ts
import {
  proxyActivities,
  proxyLocalActivities,
  ActivityFailure,
  TimeoutFailure,
} from '@temporalio/workflow';
import type { LoanActivities, LoanApplication } from './activities';

const { validateApplication } = proxyLocalActivities<LoanActivities>({  1
  startToCloseTimeout: '2 seconds',
});
const { getCreditScore } = proxyActivities<LoanActivities>({
  startToCloseTimeout: '2 seconds', // the bureau's SLA  2
  retry: { initialInterval: '1 second', maximumAttempts: 2 },
});
const { runFraudCheck, notifyApplicant } = proxyActivities<LoanActivities>({
  startToCloseTimeout: '30 seconds',
});
```

1 Cheap validation as a local activity.

2 The timeout encodes the partner's SLA; two attempts, then give up.

```ts
export type Decision =
  | { decision: 'APPROVED'; ratePct: number }
  | { decision: 'REJECTED'; reason: string }
  | { decision: 'MANUAL_REVIEW'; reason: string };

export async function loanApplicationWorkflow(
  app: LoanApplication,
): Promise<Decision> {
  await validateApplication(app); // invalid input: workflow FAILS (bad request)  3

  let score: number;
  try {
    score = await getCreditScore(app.applicantId);
  } catch (err) {
    // Infrastructure problem: degrade to a human instead of failing the application
    if (err instanceof ActivityFailure && err.cause instanceof TimeoutFailure) {  4
      return decide(app, {
        decision: 'MANUAL_REVIEW',
        reason: 'Credit bureau did not respond in time',
      });
    }
    throw err;
  }
```

3 A non-retryable `InvalidApplication` fails the execution: the request itself was wrong.

4 Infrastructure trouble is **not** the applicant's fault: route to a human instead of failing.

```
  if ((await runFraudCheck(app)) === 'HIGH') {
    return decide(app, { decision: 'MANUAL_REVIEW', reason: 'High fraud risk' });
  }
  if (score < 600) {  5
    // business rule: a result, not an error
    return decide(app, {
      decision: 'REJECTED',
      reason: `Credit score ${score} below 600`,
    });
  }
  return decide(app, { decision: 'APPROVED', ratePct: score >= 750 ? 8.5 : 11.9 });
}

async function decide(app: LoanApplication, decision: Decision): Promise<Decision> {  6
  await notifyApplicant(app.applicantId, `Your application: ${decision.decision}`);
  return decision;
}
```

5	Business rules produce decisions (data), never exceptions.
6	Plain helper functions are fine in workflows — they're just code.

What the tests prove (`test/loan-application.test.ts`, all passing):

Test	Verified behaviour
good credit is approved at the best rate	Score 780 → APPROVED, 8.5 %.
low credit is a rejection (a result, not an error)	Score 540 → REJECTED; the workflow **completes**.
high fraud risk goes to manual review	MANUAL_REVIEW.
a bureau timeout degrades to manual review instead of failing	Bureau sleeps 3 s vs. 2 s SLA, twice → MANUAL_REVIEW. (Takes ~4 s of real time: activity timeouts are not skipped while the activity runs.)
invalid input fails the workflow with a typed error	Loan > 5× income → FAILED, root `InvalidApplication`.

14. Cancellation

Cancellation is cooperative. `handle.cancel()` raises a `CancelledFailure` at the workflow's current `await`; the workflow can clean up before exiting.

```ts
import { CancellationScope, isCancellation, proxyActivities } from '@temporalio/workflow';
import type * as activities from './activities';

const { provisionVm, deleteVm, processData } = proxyActivities<typeof activities>({
  startToCloseTimeout: '10 minutes', heartbeatTimeout: '30 seconds',
});

export async function dataJobWorkflow(jobId: string): Promise<void> {
  const vmId = await provisionVm(jobId);
  try {
    await processData(vmId);
  } catch (err) {
    if (isCancellation(err)) console.log('Job cancelled, cleaning up');
    throw err;
  } finally {
    // Cleanup must run even though the workflow is being cancelled
    await CancellationScope.nonCancellable(() => deleteVm(vmId));
  }
}

// Time-box part of a workflow
await CancellationScope.withTimeout('5 minutes', () => processData(vmId));

// Cancel one branch manually
const scope = new CancellationScope();
const p = scope.run(() => processData(vmId));
scope.cancel();
```

Activities only learn about cancellation through heartbeats: call `heartbeat()` regularly and use `cancellationSignal()` or `await cancelled()` to stop work.

14.1 Real-world example: video transcoding

The situation. Uploaded videos are transcoded chunk by chunk (minutes to hours). Workers can crash mid-job, and users can cancel. Scratch storage must always be freed.

Why Temporal fits:

- Heartbeats detect a dead worker in seconds even for 4-hour jobs.
- Heartbeat details are a checkpoint: the retry resumes where the crash happened.
- Cancellation reaches the running activity; cleanup still runs.

```ts
// src/video-transcoding/activities.ts
import { activityInfo, heartbeat, cancelled, log } from '@temporalio/activity';

export interface TranscodeJob {
  videoId: string;
  chunks: number;
  resolution: string;
}

export interface Storage {
  allocate(videoId: string): Promise<string>; // scratch dir / bucket prefix
  release(workDir: string): Promise<void>;
  publish(videoId: string, workDir: string): Promise<string>;
}
export interface Encoder {
  encodeChunk(workDir: string, chunk: number, resolution: string): Promise<void>;
}
```

```
export function createVideoActivities(storage: Storage, encoder: Encoder) {
  return {
    async allocateWorkspace(videoId: string) {
      return storage.allocate(videoId);
    },
    async releaseWorkspace(workDir: string) {
      await storage.release(workDir);
    },
    async publish(videoId: string, workDir: string) {
      return storage.publish(videoId, workDir);
    },

    // Long-running: reports progress with heartbeats and resumes after a retry
    async transcode(job: TranscodeJob, workDir: string): Promise<number> {
      const resumeFrom: number = activityInfo().heartbeatDetails ?? 0;  1
      if (resumeFrom > 0) log.info('Resuming transcode', { resumeFrom });
```

1	After a crash the retry receives the **last heartbeat details**: resume from that chunk instead of chunk 0.

```
      for (let chunk = resumeFrom; chunk < job.chunks; chunk++) {
        // Stop promptly if the workflow (or a user) cancels the job
        await Promise.race([
          encoder.encodeChunk(workDir, chunk, job.resolution),  2
          cancelled(),  3
        ]);
        heartbeat(chunk + 1); // "alive" + checkpoint for retries  4
      }
      return job.chunks;
    },
  };
}

export type VideoActivities = ReturnType<typeof createVideoActivities>;
```

2	Real work (ffmpeg, an encoding API...).
3	Rejects as soon as cancellation arrives (delivered via the heartbeat response), aborting the current chunk.
4	Liveness signal **and** checkpoint. Heartbeats are throttled by the SDK, so calling it often is cheap.

```ts
// src/video-transcoding/workflows.ts
import { proxyActivities, CancellationScope } from '@temporalio/workflow';
import type { VideoActivities, TranscodeJob } from './activities';

const { allocateWorkspace, releaseWorkspace, publish } =
  proxyActivities<VideoActivities>({
    startToCloseTimeout: '1 minute',
  });
const { transcode } = proxyActivities<VideoActivities>({
  startToCloseTimeout: '4 hours', // a feature film can take hours...  1
  heartbeatTimeout: '2 seconds', // ...but a dead worker is detected in seconds  2
  retry: { initialInterval: '1 second', maximumAttempts: 5 },
});

export async function transcodeVideoWorkflow(job: TranscodeJob): Promise<string> {
  const workDir = await allocateWorkspace(job.videoId);
  try {
    await transcode(job, workDir);
    return await publish(job.videoId, workDir);
  } finally {  3
    // Runs on success, failure AND cancellation: never leak scratch storage
    await CancellationScope.nonCancellable(() => releaseWorkspace(workDir));  4
  }
}
```

1	Generous total time for legitimately long work...
2	...but no heartbeat for 2 s means the worker died: retry on another worker.
3	Cleanup lives in `finally`: success, failure or cancellation.
4	Inside a cancelled workflow, new activities would be cancelled too; `nonCancellable` lets the cleanup run.

What the tests prove (`test/video-transcoding.test.ts`, all passing):

Test	Verified behaviour
a crash mid-job resumes from the last heartbeat, not from zero	Chunk 6 crashes once; the log shows `resumeFrom: 6`; all 10 chunks encoded, far fewer than 20 calls; workspace released.
cancelling the job stops encoding and still frees the workspace	Cancel after ~1 s of a 10 s job: result is `CancelledFailure`, only part of the 100 chunks encoded, workspace released.

Tip: The worker logs that keeping heartbeat details on activity **failure** needs Temporal Server 1.16+. The time-skipping test server doesn't advertise it (the test still passed); the dev server (1.32) and Temporal Cloud guarantee it.

15. Child Workflows, Continue-As-New & Fan-out

15.1 Child workflows

```
import { executeChild, startChild, ParentClosePolicy } from '@temporalio/workflow';

export async function onboardingWorkflow(companyId: string, userIds: string[]) {
  // Wait for all children (each is its own execution with its own history)
  await Promise.all(userIds.map((uid) =>
    executeChild(onboardUserWorkflow, {
      workflowId: `onboard-${companyId}-${uid}`,
      args: [uid],
    })));

  // Start a child that outlives the parent
  await startChild(auditWorkflow, {
    workflowId: `audit-${companyId}`, args: [companyId],
    parentClosePolicy: ParentClosePolicy.ABANDON,
  });
}
```

Use children to partition big work (each child has its own 50K-event history limit), to reuse workflows, or to give a sub-process its own ID and lifecycle.

15.2 Continue-As-New

A single execution's history is limited (warnings at ~10K events, hard limit ~50K events / 50 MB). Long-running or looping workflows should periodically restart themselves with fresh history, carrying state forward as arguments:

```
import { continueAsNew, workflowInfo, condition, allHandlersFinished }
  from '@temporalio/workflow';

export async function accountWorkflow(state: AccountState): Promise<void> {
  setupHandlers(state);                              // signals mutate state
  while (!workflowInfo().continueAsNewSuggested) {
    await condition(() => state.pending.length > 0, '1 hour');
    await processPending(state);
  }
  await condition(allHandlersFinished);              // don't drop in-flight signals
  await continueAsNew<typeof accountWorkflow>(state);
}
```

15.3 Batch fan-out with bounded concurrency

```
export async function batchWorkflow(ids: string[]): Promise<number> {
  const CHUNK = 50;
  let ok = 0;
  for (let i = 0; i < ids.length; i += CHUNK) {
    const results = await Promise.allSettled(
      ids.slice(i, i + CHUNK).map((id) => processItem(id)));
    ok += results.filter((r) => r.status === 'fulfilled').length;
  }
  return ok;
}
// For 10K+ items: fan out to child workflows, or continue-as-new every N chunks.
```

15.4 Real-world example: employee onboarding

The situation. A new hire needs accounts (IT), a laptop (Facilities) and payroll (Finance), plus a records clean-up 30 days later. Departments own their parts.

Why Temporal fits:

- Child workflows give each department its own execution, history and UI page.

- Children run in parallel; a failing child is handled, not fatal.
- `ParentClosePolicy.ABANDON` lets the 30-day child outlive the parent.

```ts
// src/onboarding/workflows.ts
import {
  proxyActivities,
  executeChild,
  startChild,
  workflowInfo,
  sleep,
  ApplicationFailure,
  ChildWorkflowFailure,
  ParentClosePolicy,
} from '@temporalio/workflow';
import type { OnboardingActivities, Employee } from './activities';

const a = proxyActivities<OnboardingActivities>({
  startToCloseTimeout: '1 minute',
});
```

```ts
export interface OnboardingSummary {
  employeeId: string;
  accounts: string[];
  laptopOrder: string;
  payroll: string;
}

// Parent: coordinates one child workflow per department
export async function onboardEmployeeWorkflow(
  emp: Employee,
): Promise<OnboardingSummary> {
  const base = workflowInfo().workflowId; // e.g. onboard-emp-42  1

  const [accounts, laptopOrder] = await Promise.all([  2
    executeChild(provisionAccountsWorkflow, {
      workflowId: `${base}-it`,
      args: [emp],
    }),
    executeChild(shipLaptopWorkflow, { workflowId: `${base}-laptop`, args: [emp] }),
  ]);
```

1. Derive child IDs from the parent ID: unique, readable, and idempotent.
2. IT and Facilities run as child workflows **in parallel**.

```
  let payroll: string;
  try {
    payroll = await executeChild(setupPayrollWorkflow, {
      workflowId: `${base}-payroll`,
      args: [emp],
    });
  } catch (err) {
    if (!(err instanceof ChildWorkflowFailure)) throw err;  3
    // One department failing must not undo the rest: hand it to a human
    await a.notifyHr(emp.id, `Payroll setup failed: ${err.cause?.message}`);
    payroll = 'PENDING_HR';  4
  }

  // Fire-and-forget child that outlives the parent (runs for 30 days)
  await startChild(retentionWorkflow, {
    workflowId: `${base}-retention`,
    args: [emp.id],
    parentClosePolicy: ParentClosePolicy.ABANDON,  5
  });
```

3	A child's failure arrives as `ChildWorkflowFailure` with the original error as `.cause`.
4	Contain the failure: onboarding continues, HR finishes payroll by hand.
5	This child keeps running after the parent completes (default would terminate it).

```
  return { employeeId: emp.id, accounts, laptopOrder, payroll };
}

// IT: three systems in parallel
export async function provisionAccountsWorkflow(emp: Employee): Promise<string[]> {  6
  return Promise.all([
    a.createEmailAccount(emp),
    a.inviteToSlack(emp),
    a.addToGitHub(emp),
  ]);
}

// Facilities
export async function shipLaptopWorkflow(emp: Employee): Promise<string> {
  return a.orderLaptop(
    emp.id,
    emp.role === 'engineer' ? 'MacBook Pro 16' : 'MacBook Air 13',
  );
}
```

6	Each child has its own history, retries and UI page — IT can inspect its part independently.

```
// Finance
export async function setupPayrollWorkflow(emp: Employee): Promise<string> {
  if (!emp.bankAccount) {
    throw ApplicationFailure.nonRetryable(
      'Missing bank details',
      'MissingBankDetails',
    );
  }
  return a.registerPayroll(emp.id, emp.bankAccount);
}

// Compliance: archive onboarding records after the probation check-in
export async function retentionWorkflow(employeeId: string): Promise<void> {
  await sleep('30 days');  7
  await a.archiveOnboardingRecords(employeeId);
}
```

7	A 30-day durable timer in a child that nobody waits for.

What the tests prove (`test/onboarding.test.ts`, all passing):

Test	Verified behaviour
departments run as child workflows; the parent combines results	Summary has all accounts, the right laptop and payroll; `...-it` is COMPLETED, `...-retention` is still RUNNING.
a failing child is handled by the parent, onboarding continues	Missing bank details → payroll PENDING_HR, HR notified with the root cause.

15.5 Real-world example: a loyalty-points account (entity workflow)

The situation. Every customer has a points balance and tier. Orders arrive as events (with occasional duplicates), customers redeem points, and the account lives for years.

Why Temporal fits:

- One workflow per customer is the account: no separate state table.
- Updates validate redemptions; queries serve the balance.
- Continue-as-new keeps the history small forever.

```ts
// src/loyalty/workflows.ts
import {
  proxyActivities,
  defineSignal,
  defineQuery,
  defineUpdate,
  setHandler,
  condition,
  continueAsNew,
  workflowInfo,
  allHandlersFinished,
} from '@temporalio/workflow';
import type { LoyaltyActivities, Tier } from './activities';

const { sendTierChangeEmail } = proxyActivities<LoyaltyActivities>({
  startToCloseTimeout: '1 minute',
});
```

```ts
export interface LoyaltyState {
  customerId: string;
  points: number;
  tier: Tier;
  recentOrders: string[]; // de-duplication window for redelivered events  1
  maxEventsPerRun?: number; // continue-as-new threshold (default 500)
}

export const earnPoints =
  defineSignal<[{ orderId: string; amountCents: number }]>('earn');
export const redeemPoints = defineUpdate<number, [number]>('redeem');
export const getAccount = defineQuery<{
  points: number;
  tier: Tier;
  eventsThisRun: number;
}>('getAccount');
```

1 Event sources (Kafka, webhooks) redeliver: remember recent IDs to stay idempotent.

```
const tierFor = (points: number): Tier =>
  points >= 1000 ? 'GOLD' : points >= 100 ? 'SILVER' : 'BRONZE';

// "Entity workflow": one long-lived workflow per customer account (Workflow ID  2
// loyalty-<customerId>). It never completes; it continues-as-new to stay small.
export async function loyaltyAccountWorkflow(state: LoyaltyState): Promise<void> {
  const maxEvents = state.maxEventsPerRun ?? 500;
  let events = 0;

  setHandler(earnPoints, ({ orderId, amountCents }) => {
    if (state.recentOrders.includes(orderId)) return; // duplicate delivery: ignore  3
    state.points += Math.floor(amountCents / 100); // 1 point per dollar
    state.recentOrders = [...state.recentOrders, orderId].slice(-200);
    events++;
  });
```

2 The workflow **is** the account: state, rules and side effects in one place, one ID per customer.

3 Duplicate event: ignored (the test sends one on purpose).

```
  setHandler(
    redeemPoints,
    (points) => {
      state.points -= points;
      events++;
      return state.points;
    },
    {
      validator: (points) => {  4
        if (points <= 0 || points > state.points) {
          throw new Error(
            `Cannot redeem ${points} points (balance ${state.points})`,
          );
        }
      },
    },
  );
```

4 Redeeming more than the balance is rejected before it is recorded.

```
  setHandler(getAccount, () => ({
    points: state.points,
    tier: state.tier,
    eventsThisRun: events,
  }));

  for (;;) {
    await condition(
      () =>
        tierFor(state.points) !== state.tier ||
        events >= maxEvents ||
        workflowInfo().continueAsNewSuggested,
    );

    const tier = tierFor(state.points);
    if (tier !== state.tier) {
      const previous = state.tier;
      state.tier = tier;
      await sendTierChangeEmail(state.customerId, previous, tier);  5
    }
```

5 Side effect on a state transition (BRONZE → SILVER).

```
    if (events >= maxEvents || workflowInfo().continueAsNewSuggested) {  6
      await condition(allHandlersFinished); // don't drop in-flight updates
      await continueAsNew<typeof loyaltyAccountWorkflow>(state); // fresh history  7
    }
  }
}
```

6 Histories are capped (~50K events). Restart periodically, or when the server suggests it.

7 Same Workflow ID, new run, empty history, state carried as input.

```
// test/loyalty.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { WorkflowUpdateFailedError } from '@temporalio/client';
import { createLoyaltyActivities } from '../src/loyalty/activities';
import {
  loyaltyAccountWorkflow,
  earnPoints,
  redeemPoints,
  getAccount,
} from '../src/loyalty/workflows';
import { startWorker, uid, RunningWorker } from './helpers';

let env: TestWorkflowEnvironment;
let worker: RunningWorker;
const mails: string[] = [];
```

```
before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();
  worker = await startWorker(
    env,
    'loyalty',
    require.resolve('../src/loyalty/workflows'),
    createLoyaltyActivities({
      send: async (c, t) => {
        mails.push(`${c}:${t}`);
      },
    }),
  );
});
after(async () => {
  await worker.stop();
  await env.teardown();
});
```

```
test('one workflow per customer, continued-as-new to stay small', async () => {
  const workflowId = `loyalty-${uid('cus')}`;
  const handle = await env.client.workflow.start(loyaltyAccountWorkflow, {
    taskQueue: 'loyalty',
    workflowId,
    args: [
      {
        customerId: 'cus_7',
        points: 0,
        tier: 'BRONZE',
        recentOrders: [],
        maxEventsPerRun: 5,  1
      },
    ], // tiny threshold for the test
  });
  const firstRunId = (await handle.describe()).runId;
```

1 Tiny threshold so the test crosses it.

```
  for (let i = 1; i <= 12; i++) {
    // $10 order = +10 points
    await handle.signal(earnPoints, { orderId: `ord_${i}`, amountCents: 1000 });
  }
  // the same order event delivered twice
  await handle.signal(earnPoints, { orderId: 'ord_3', amountCents: 1000 });  2

  // Continue-as-new is asynchronous (after the tier e-mail): wait for the new run  3
  for (let i = 0; i < 50 && (await handle.describe()).runId === firstRunId; i++) {
    await env.sleep(100);
  }
  assert.notEqual((await handle.describe()).runId, firstRunId, 'continued as new');

  // Queries and updates on the same handle follow the latest run automatically
  const account = await handle.query(getAccount);
  assert.equal(account.points, 120); // state carried over, duplicate ignored
  assert.equal(account.tier, 'SILVER');
  assert.equal(account.eventsThisRun, 0, 'the new run starts with a fresh history');  4
  assert.deepEqual(mails, ['cus_7:tier-bronze-to-silver']);
```

2	The duplicate: the balance must not change.
3	Observed: a burst of signals can be delivered in **one** workflow task, so the first run processed all 12 before continuing as new.
4	Points and tier survived; the per-run counter started over.

```
  await assert.rejects(
    handle.executeUpdate(redeemPoints, { args: [500] }),
    WorkflowUpdateFailedError,
  );
  assert.equal(await handle.executeUpdate(redeemPoints, { args: [50] }), 70);
  await handle.terminate('test done');
});
```

What the tests prove (`test/loyalty.test.ts`, all passing):

Test	Verified behaviour
one workflow per customer, continued-as-new to stay small	12 orders + 1 duplicate → 120 points, SILVER, one tier e-mail; new run ID; counter reset; over-redeem rejected; redeem 50 → 70.

15.6 Real-world example: an e-mail campaign

The situation. Marketing sends a campaign to a mailing list. The provider allows limited concurrency, some addresses are invalid, and lists can have millions of entries.

Why Temporal fits:

- Bounded fan-out: batches of parallel activities.
- Pagination + continue-as-new handle lists of any size.
- A query gives the dashboard live progress.

```ts
// src/email-campaign/workflows.ts
import {
  proxyActivities,
  defineQuery,
  setHandler,
  continueAsNew,
  workflowInfo,
} from '@temporalio/workflow';
import type { CampaignActivities } from './activities';

const { loadRecipients } = proxyActivities<CampaignActivities>({
  startToCloseTimeout: '1 minute',
});
const { sendCampaignEmail } = proxyActivities<CampaignActivities>({
  startToCloseTimeout: '30 seconds',
  retry: { initialInterval: '5 seconds', maximumAttempts: 5 }, // provider rate limits
});
```

```ts
export interface CampaignInput {
  campaignId: string;
  listId: string;
  template: string;
  batchSize?: number; // emails in flight at once (protects the provider)
  pageSize?: number; // recipients loaded per page
  offset?: number; // cursor, carried across continue-as-new  1
  sent?: number;
  failed?: string[];
}

export interface CampaignProgress {
  sent: number;
  failed: number;
  offset: number;
}
export const progress = defineQuery<CampaignProgress>('progress');  2
```

1 Progress lives in the input so a continued-as-new run picks up exactly here.

2 A live progress bar for the marketing dashboard.

```
export async function emailCampaignWorkflow(input: CampaignInput) {
  const { listId, template, batchSize = 50, pageSize = 500 } = input;
  let offset = input.offset ?? 0;
  let sent = input.sent ?? 0;
  const failed = input.failed ?? [];
  setHandler(progress, () => ({ sent, failed: failed.length, offset }));

  for (;;) {
    const page = await loadRecipients(listId, offset, pageSize);  3
    if (page.length === 0) break;

    // Bounded concurrency: one batch at a time, every email in the batch in parallel
    for (let i = 0; i < page.length; i += batchSize) {
      const batch = page.slice(i, i + batchSize);
      const results = await Promise.allSettled(  4
        batch.map((r) => sendCampaignEmail(r, template)),
      );
      results.forEach((r, j) => {
        if (r.status === 'fulfilled') sent++;
        else failed.push(batch[j].email); // bad address: record and move on  5
      });
    }
    offset += page.length;
```

3	Load recipients page by page: payloads stay small (each is limited to 2 MB).
4	One batch in flight at a time protects the e-mail provider's rate limits.
5	Invalid addresses are non-retryable: recorded, not retried, not fatal.

```
    // Huge lists: start a fresh run (new history) with the cursor carried over
    if (workflowInfo().continueAsNewSuggested) {  6
      await continueAsNew<typeof emailCampaignWorkflow>({
        ...input,
        offset,
        sent,
        failed,
      });
    }
  }
  return { campaignId: input.campaignId, sent, failed };
}
```

6	1 million recipients would exceed one history; continue-as-new with the cursor.

What the tests prove (`test/email-campaign.test.ts`, all passing):

Test	Verified behaviour
sends to every valid address in bounded batches, skips bad ones	120 recipients, 3 invalid: 117 sent, 3 failed, nobody twice, never more than 10 in flight, progress query correct.

Part IV — Common Patterns

16. Patterns Cookbook

16.1 Real-world example: booking a trip (saga)

The situation. A trip needs a flight, a hotel and a rental car from three different suppliers. If the car is sold out, the flight and hotel must be cancelled — even if the hotel's cancellation API is flaky.

Why Temporal fits:

- No distributed transactions across suppliers: compensate instead.
- Compensations are activities with their own (unlimited) retry policy.
- The order of undo is guaranteed by plain code.

```
// src/trip-booking/workflows.ts
import {
  proxyActivities,
  ApplicationFailure,
  CancellationScope,
  log,
} from '@temporalio/workflow';
import type { TripActivities, Trip } from './activities';

const a = proxyActivities<TripActivities>({
  startToCloseTimeout: '30 seconds',
  retry: { initialInterval: '1 second', maximumAttempts: 3 },
});
// Compensations must eventually succeed: retry them (much) longer
const undo = proxyActivities<TripActivities>({  1
  startToCloseTimeout: '30 seconds',
  retry: { initialInterval: '1 second', maximumInterval: '1 minute' }, // unlimited  2
});
```

1 Compensations get their own, more persistent retry policy.

2 No `maximumAttempts`: a cancellation must eventually succeed (the test makes it fail twice).

```
export interface Itinerary {
  flight: string;
  hotel: string;
  car: string;
}

// Saga: a "distributed transaction" across three suppliers without 2-phase commit
export async function bookTripWorkflow(trip: Trip): Promise<Itinerary> {
  const compensations: Array<() => Promise<void>> = [];
  try {
    const flight = await a.bookFlight(trip);
    compensations.unshift(() => undo.cancelFlight(flight));  3

    const hotel = await a.bookHotel(trip);
    compensations.unshift(() => undo.cancelHotel(hotel));
```

3 Register the undo only **after** the step succeeded; `unshift` builds reverse order.

```
    const car = await a.bookCar(trip);
    compensations.unshift(() => undo.cancelCar(car));

    return { flight, hotel, car };
  } catch (err) {
    log.warn('Trip booking failed, compensating', { steps: compensations.length });
    await CancellationScope.nonCancellable(async () => {  4
      for (const compensate of compensations) await compensate(); // reverse order  5
    });
    throw ApplicationFailure.nonRetryable(
      `Trip ${trip.tripId} could not be booked and was rolled back`,
      'TripRolledBack',  6
    );
  }
}
```

4	Compensation must run even if someone cancels the booking workflow.
5	Hotel is released before the flight: last booked, first undone.
6	The workflow still fails — but cleanly, with nothing left booked.

What the tests prove (`test/trip-booking.test.ts`, all passing):

Test	Verified behaviour
all three suppliers confirm: the trip is booked	Flight, hotel, car booked in order.
car sold out: hotel and flight are cancelled in reverse order	Journal is book flight, book hotel, cancel hotel, cancel flight; the hotel cancellation failed twice and was retried; workflow fails with "rolled back".

16.2 Real-world example: expense approval with escalation

The situation. Employees submit expenses. Small ones are auto-approved; others need the manager within 2 days, then the director within 3 more, else they are auto-rejected.

Why Temporal fits:

- Days-long waits are just `condition` with a timeout.
- Signals carry decisions from the finance UI; a query shows who is blocking.
- The whole policy is readable top-to-bottom in one function.

```
// src/expense-approval/workflows.ts
import {
  proxyActivities,
  defineSignal,
  defineQuery,
  setHandler,
  condition,
} from '@temporalio/workflow';
import type { ExpenseActivities, Expense } from './activities';

const a = proxyActivities<ExpenseActivities>({ startToCloseTimeout: '1 minute' });

export interface Decision {
  approved: boolean;
  by: string;
  comment?: string;
}
export const decide = defineSignal<[Decision]>('decide'); // from the approval UI  1
export const approvalState = defineQuery<string>('approvalState');
```

1	The approve/reject buttons in the finance tool send this signal.

```
export type ExpenseOutcome =
  | { status: 'PAID'; payoutId: string; approvedBy: string }
  | { status: 'REJECTED'; by: string; comment?: string }
  | { status: 'AUTO_REJECTED' };

const AUTO_APPROVE_LIMIT = 50;

export async function expenseApprovalWorkflow(
  expense: Expense,
): Promise<ExpenseOutcome> {
  let decision: Decision | undefined;
  let state = 'SUBMITTED';
  setHandler(decide, (d) => {
    decision ??= d;  2
  }); // first decision wins
  setHandler(approvalState, () => state);
```

2 First decision wins; a late second click is ignored.

```
  if (expense.amount <= AUTO_APPROVE_LIMIT) {  3
    // policy: small expenses auto-approved
    decision = { approved: true, by: 'policy' };
  } else {
    // Level 1: the manager has 2 days
    state = 'WAITING_FOR_MANAGER';
    await a.requestApproval(expense.managerEmail, expense);
    if (!(await condition(() => decision !== undefined, '2 days'))) {  4
      // Level 2: escalate to the director for 3 more days
      state = 'ESCALATED_TO_DIRECTOR';  5
      await a.requestApproval(expense.directorEmail, expense);
      if (!(await condition(() => decision !== undefined, '3 days'))) {
        state = 'AUTO_REJECTED';
        await a.notifyEmployee(
          expense.employee,
          'Expense auto-rejected: no decision in 5 days',
        );
        return { status: 'AUTO_REJECTED' };  6
      }
    }
  }
```

3 Policy before people: small amounts never wait for a human.

4 The manager has 2 days...

5 ...then the director gets it. The query lets the UI show who is holding it up.

6 Nobody decided in 5 days: close it rather than wait forever.

```
  if (!decision!.approved) {
    state = 'REJECTED';
    await a.notifyEmployee(
      expense.employee,
      `Expense rejected: ${decision!.comment ?? ''}`,
    );
    return { status: 'REJECTED', by: decision!.by, comment: decision!.comment };
  }
  state = 'APPROVED';
  const payoutId = await a.reimburse(expense.employee, expense.amount);
  state = 'PAID';
  return { status: 'PAID', payoutId, approvedBy: decision!.by };
}
```

What the tests prove (`test/expense-approval.test.ts`, all passing):

Test	Verified behaviour
small expenses are approved by policy without bothering anyone	35 → PAID by "policy".
the manager approves within 2 days	State WAITING_FOR_MANAGER, then PAID.
a silent manager escalates to the director, who rejects	After 49 h: ESCALATED_TO_DIRECTOR; rejection recorded; notifications to manager, director, employee.
nobody decides within 5 days: auto-rejected	AUTO_REJECTED.

16.3 Real-world example: identity verification (polling)

The situation. A fintech submits a customer's documents to a KYC provider that has no webhooks. The verdict can take minutes or days; after a deadline the case goes to manual review.

Why Temporal fits:

- The retry policy *is* the polling loop: fixed interval, total deadline.
- No sleeping activities, no hand-written loops.
- One subtle detail about deadlines, found by the test (callout 4).

```ts
// src/kyc-verification/activities.ts
import { ApplicationFailure, activityInfo } from '@temporalio/activity';

export type KycStatus = 'PENDING' | 'VERIFIED' | 'REJECTED';

// Third-party identity verification provider (Onfido, Jumio, Persona, ...)
export interface KycProvider {
  // returns the provider's case ID
  submit(customerId: string, documents: string[]): Promise<string>;
  status(caseId: string): Promise<KycStatus>;
}

export function createKycActivities(provider: KycProvider) {
  return {
    async submitDocuments(customerId: string, documents: string[]) {
      return provider.submit(customerId, documents);
    },
    // Polling via retries: throw while the result isn't ready; the retry policy
    // decides how often to ask and when to give up
    async waitForVerdict(caseId: string): Promise<'VERIFIED' | 'REJECTED'> {  1
      const status = await provider.status(caseId);
      if (status === 'PENDING') {
        throw ApplicationFailure.retryable(  2
          `Case ${caseId} still pending (poll #${activityInfo().attempt})`,
          'StillPending',
        );
      }
      return status;
    },
  };
}

export type KycActivities = ReturnType<typeof createKycActivities>;
```

1	One status check per attempt; the activity itself never loops or sleeps.
2	"Not ready yet" = a retryable error. The retry policy becomes the polling loop.

```ts
// src/kyc-verification/workflows.ts
import {
  proxyActivities,
  ActivityFailure,
  TimeoutFailure,
} from '@temporalio/workflow';
import { RetryState, type Duration } from '@temporalio/common';
import type { KycActivities } from './activities';

const { submitDocuments } = proxyActivities<KycActivities>({
  startToCloseTimeout: '1 minute',
});

export interface KycInput {
  customerId: string;
  documents: string[];
  pollEvery?: Duration; // how often to ask the provider (default 1 minute)
  giveUpAfter?: Duration; // how long to wait for a verdict (default 2 days)
}
```

```ts
export type KycOutcome = 'VERIFIED' | 'REJECTED' | 'MANUAL_REVIEW';

export async function kycVerificationWorkflow(
  input: KycInput,
): Promise<KycOutcome> {
  const caseId = await submitDocuments(input.customerId, input.documents);

  // Proxies can be created inside the workflow: options come from the input
  const { waitForVerdict } = proxyActivities<KycActivities>({  1
    startToCloseTimeout: '30 seconds', // one status call
    scheduleToCloseTimeout: input.giveUpAfter ?? '2 days', // all polls together  2
    retry: {
      initialInterval: input.pollEvery ?? '1 minute',
      backoffCoefficient: 1, // fixed interval: a steady poll  3
    },
  });
```

1. Proxies may be created inside the workflow, here with options from the input.
2. Total budget across all polls.
3. Coefficient 1 = poll every `pollEvery` (no exponential growth).

```ts
  try {
    return await waitForVerdict(caseId);
  } catch (err) {
    // Deadline hit between polls: the cause is the LAST attempt's error and
    // retryState says TIMEOUT. Deadline hit during a poll: cause is TimeoutFailure.
    if (
      err instanceof ActivityFailure &&
      (err.retryState === RetryState.TIMEOUT || err.cause instanceof TimeoutFailure)  4
    ) {
      return 'MANUAL_REVIEW'; // provider never answered
    }
    throw err;
  }
}
```

4. Found by the test: a deadline that expires **between** polls returns the last error with `retryState TIMEOUT`, not a `TimeoutFailure`.

What the tests prove (`test/kyc-verification.test.ts`, all passing):

Test	Verified behaviour
polls the provider every minute until the verdict arrives	3 × PENDING then VERIFIED: exactly 4 polls.
no verdict before the deadline: falls back to manual review	10-minute deadline, 1-minute interval: ~10 polls, then MANUAL_REVIEW.

16.4 Real-world example: human document review (async completion)

The situation. Uploaded contracts are reviewed by a person in an external back-office tool. The tool calls a webhook when the reviewer decides — maybe days later.

Why Temporal fits:

- The activity hands its task token to the tool and returns without a result.
- The webhook completes (or fails) that exact activity from outside.
- No polling, no worker slot held while humans think.

```ts
// src/document-review/activities.ts
import { activityInfo, CompleteAsyncError } from '@temporalio/activity';

export interface ReviewResult {
  approved: boolean;
  reviewer: string;
  notes?: string;
}

// External review tool (Zendesk, Jira, an internal back-office app, ...)
export interface ReviewQueue {
  createTask(documentId: string, callbackToken: string): Promise<void>;
}
export interface Archive {
  store(documentId: string, status: string): Promise<void>;
}
```

```ts
export function createReviewActivities(queue: ReviewQueue, archive: Archive) {
  return {
    // Hands the work to humans and returns WITHOUT a result: the review tool's
    // webhook completes this activity later (minutes or days) using the token
    async requestHumanReview(documentId: string): Promise<ReviewResult> {
      const token = Buffer.from(activityInfo().taskToken).toString('base64url');  // 1
      await queue.createTask(documentId, token);  // 2
      throw new CompleteAsyncError();  // 3
    },
    async archiveDocument(documentId: string, status: string) {
      await archive.store(documentId, status);
    },
  };
}

export type ReviewActivities = ReturnType<typeof createReviewActivities>;
```

1. The task token identifies this exact activity attempt; base64url makes it URL-safe for a webhook.
2. Hand the token to the external system along with the work.
3. "I'll be completed later": the activity stays open without occupying a worker slot.

```ts
// src/document-review/webhook.ts
import type { Client } from '@temporalio/client';
import type { ReviewResult } from './activities';

// Called by the review tool's webhook (e.g. a Fastify/Express route) when a
// reviewer clicks Approve / Reject. The token came from requestHumanReview().
export async function onReviewSubmitted(
  client: Client,
  token: string,
  result: ReviewResult,
) {
  await client.activity.complete(Buffer.from(token, 'base64url'), result);  1
}

// The reviewer tool can also report that the document is unreadable
export async function onReviewImpossible(
  client: Client,
  token: string,
  reason: string,
) {
  await client.activity.fail(Buffer.from(token, 'base64url'), new Error(reason));  2
}
```

1	Completes the waiting activity with a result; the workflow continues.
2	Or fails it; the workflow sees an `ActivityFailure`.

What the tests prove (`test/document-review.test.ts`, all passing):

Test	Verified behaviour
reviewer approves via webhook: the waiting activity completes	Result APPROVED; document archived.
the reviewer reports the document as unreadable	`client.activity.fail` → workflow handles it → UNREADABLE.

16.5 Entity workflow (one workflow per long-lived object)

Model a customer, device or subscription as a workflow with ID `subscription-{id}`. Signals/updates change it, queries read it, a loop with `condition` waits for events, and continue-as-new keeps history small (Chapter 15.2 and the loyalty example in 15.5). Monthly billing becomes:

```ts
export async function subscriptionWorkflow(s: Sub): Promise<void> {
  setHandler(cancelSub, () => { s.cancelled = true; });
  for (let i = 0; i < 12 && !s.cancelled; i++) {
    await chargeMonthly(s.customerId, s.plan);
    await condition(() => s.cancelled, '30 days');   // sleep, but wake on cancel
  }
  if (!s.cancelled) await continueAsNew<typeof subscriptionWorkflow>(s);
}
```

16.6 Idempotency checklist

- Use business IDs as Workflow IDs `order-{id}` — duplicate starts are rejected by the server.
- Pass an idempotency key (workflowId or `workflowId + activityId`) to payment and other external APIs.
- Use upserts / `ON CONFLICT DO NOTHING` in database activities.

Part V — Production Readiness

17. Versioning Workflow Code

Running executions replay using the *new* code you deploy. If the new code issues different commands than recorded in history, the worker raises a `DeterminismViolationError` and the workflow task gets stuck.

Safe changes	Unsafe without versioning
Activity implementation code	Adding/removing/reordering activity calls
Timeouts and retry policies	Adding/removing timers (sleep) or child workflows
Adding new signal/query/update handlers	Changing activity type names
Logging	Changing branching that alters which commands run

17.1 Patching

```
import { patched, deprecatePatch } from '@temporalio/workflow';

export async function orderWorkflow(order: Order) {
  await reserveInventory(order);
  if (patched('fraud-check-v1')) {
    await fraudCheck(order);          // new executions (and new replays) run this
  }
  await chargeCard(order);            // old executions skip it on replay
}

// Later, once NO execution started before the patch is still running:
//   1. replace patched(...) with deprecatePatch('fraud-check-v1') and keep the new code
//   2. after those finish too, delete the deprecatePatch call entirely
```

17.2 Other strategies

- **New workflow type**: copy to `orderWorkflowV2`, route new starts to it, let old ones drain.
- **Worker Versioning**: tag workers with a build ID/deployment version so executions stay pinned to the code they started on.
- **Replay tests** in CI catch non-determinism before deploy (Chapter 18.4).

17.3 Real-world example: adding a fraud check to live checkouts

The situation. Checkout reserves stock, waits for the customer's 3-D Secure confirmation, then charges. Risk wants a fraud check before charging — deployed while orders are waiting.

Why Temporal fits:

- Replay tests prove a change is safe **before** deploying.
- `patched()` lets old and new histories coexist; `deprecatePatch()` cleans up later.
- The deploy test shows what really happens to in-flight orders.

v1 (in production)

```ts
// src/versioning/workflows-v1.ts
// v1 - in production today
import {
  proxyActivities,
  defineSignal,
  setHandler,
  condition,
} from '@temporalio/workflow';
import type { CheckoutActivities } from './activities';

const a = proxyActivities<CheckoutActivities>({ startToCloseTimeout: '1 minute' });

export const confirmPayment = defineSignal('confirmPayment'); // 3-D Secure completed

export async function checkoutWorkflow(orderId: string): Promise<string> {
  let confirmed = false;
  setHandler(confirmPayment, () => {
    confirmed = true;
  });

  await a.reserveStock(orderId);
  await condition(() => confirmed); // can take minutes: customer is in their bank app
  return a.chargeCard(orderId);
}
```

v2, done wrong

```ts
// src/versioning/workflows-unsafe.ts
// v2 done WRONG - a new activity call inserted without versioning
import {
  proxyActivities,
  defineSignal,
  setHandler,
  condition,
} from '@temporalio/workflow';
import type { CheckoutActivities } from './activities';

const a = proxyActivities<CheckoutActivities>({ startToCloseTimeout: '1 minute' });

export const confirmPayment = defineSignal('confirmPayment');

export async function checkoutWorkflow(orderId: string): Promise<string> {
  let confirmed = false;
  setHandler(confirmPayment, () => {
    confirmed = true;
  });
  await a.reserveStock(orderId);
  await condition(() => confirmed);
  await a.runFraudCheck(orderId); // breaks every order that started on v1  1
  return a.chargeCard(orderId);
}
```

1 On replay of a v1 history the worker expects `chargeCard` here but the code schedules a fraud check: `DeterminismViolationError`.

v2, done right

```ts
// src/versioning/workflows-v2.ts
// v2 done RIGHT - the new step is guarded by a patch marker
import {
  proxyActivities,
  defineSignal,
  setHandler,
  condition,
  patched,
} from '@temporalio/workflow';
import type { CheckoutActivities } from './activities';

const a = proxyActivities<CheckoutActivities>({ startToCloseTimeout: '1 minute' });

export const confirmPayment = defineSignal('confirmPayment');
```

```ts
export async function checkoutWorkflow(orderId: string): Promise<string> {
  let confirmed = false;
  setHandler(confirmPayment, () => {
    confirmed = true;
  });

  await a.reserveStock(orderId);
  await condition(() => confirmed);
  if (patched('fraud-check')) {  1
    // true when this line runs for the FIRST time (new orders, and in-flight
    // orders reaching it after the deploy); false when replaying a history
    // that already passed this point without the patch  2
    await a.runFraudCheck(orderId);
  }
  return a.chargeCard(orderId);
}
```

1. `patched()` writes a marker into the history the first time it returns true; replays read that marker.
2. Verified by the deploy test: an order waiting *before* this line when v2 was deployed still gets the fraud check.

v3, cleanup

```ts
// src/versioning/workflows-v3.ts
// v3 - cleanup, deployed once NO v1 order is still running
import {
  proxyActivities,
  defineSignal,
  setHandler,
  condition,
  deprecatePatch,
} from '@temporalio/workflow';
import type { CheckoutActivities } from './activities';

const a = proxyActivities<CheckoutActivities>({ startToCloseTimeout: '1 minute' });

export const confirmPayment = defineSignal('confirmPayment');
```

```ts
export async function checkoutWorkflow(orderId: string): Promise<string> {
  let confirmed = false;
  setHandler(confirmPayment, () => {
    confirmed = true;
  });

  await a.reserveStock(orderId);
  await condition(() => confirmed);
  deprecatePatch('fraud-check'); // still accepts v2 histories (with the marker)  1
  await a.runFraudCheck(orderId); // the new path is now unconditional
  return a.chargeCard(orderId);
}
// v4 (later, when no v2 order is running): delete the deprecatePatch() line
```

1 Step 2 of the cleanup: accept histories with the marker, always run the new code.

```ts
// test/versioning.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { Worker } from '@temporalio/worker';
import { createCheckoutActivities } from '../src/versioning/activities';
import { checkoutWorkflow, confirmPayment } from '../src/versioning/workflows-v1';
import { startWorker, uid } from './helpers';

const journal: string[] = [];
const activities = createCheckoutActivities(journal);
const code = (version: string) =>
  require.resolve(`../src/versioning/workflows-${version}`);

let env: TestWorkflowEnvironment;
before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();
});
after(async () => {
  await env.teardown();
});
```

```ts
const steps = (orderId: string) =>
  journal.filter((j) => j.startsWith(`${orderId}:`)).map((j) => j.split(':')[1]);

async function waitForStep(orderId: string, step: string) {
  for (let i = 0; i < 100 && !steps(orderId).includes(step); i++)
    await env.sleep(100);
}

// Runs one complete order on the given code version and returns its event history
async function historyOf(version: string) {
  const worker = await startWorker(
    env,
    `history-${version}`,
    code(version),
    activities,
  );
  const orderId = uid(`order-${version}`);
  const handle = await env.client.workflow.start(checkoutWorkflow, {
    taskQueue: `history-${version}`,
    workflowId: orderId,
    args: [orderId],
  });
  await handle.signal(confirmPayment);
  await handle.result();
  await worker.stop();
  return handle.fetchHistory();
}
```

```
test('replay tests catch an unsafe change before it reaches production', async () => {
  const v1History = await historyOf('v1');
  await Worker.runReplayHistory({ workflowsPath: code('v2') }, v1History); // safe  1
  await assert.rejects(
    Worker.runReplayHistory({ workflowsPath: code('unsafe') }, v1History),  2
    (err: Error) => err.name === 'DeterminismViolationError',
  );
});

test('deploy mid-order: the order takes the new path when it gets there', async () => {
  // No workflow cache: like a fresh deployment, every task replays the full history
  const fresh = { maxCachedWorkflows: 0 };  3
  // Order A starts on v1 and waits for the customer's bank confirmation
  let worker = await startWorker(env, 'checkout', code('v1'), activities, fresh);
  const a = await env.client.workflow.start(checkoutWorkflow, {
    taskQueue: 'checkout',
    workflowId: uid('order-A'),
    args: ['A'],
  });
  await waitForStep('A', 'reserve');
  await worker.stop(); // ...deploy...
  worker = await startWorker(env, 'checkout', code('v2'), activities, fresh);
```

1	Replay a real v1 history against the new code: passes.
2	The unsafe change is caught **in CI**, before any production order breaks.
3	Without a cache every task replays from history, like a freshly deployed worker. (The time-skipping server doesn't reroute sticky tasks from a stopped worker; a real server does after ~5 s.)

```
  // A replays its v1 history (reserve) on v2, then runs patched() for the FIRST time:
  // the marker is recorded and the new step executes. No non-determinism.
  await a.signal(confirmPayment);
  assert.equal(await a.result(), 'ch_A');
  assert.deepEqual(steps('A'), ['reserve', 'fraud-check', 'charge']);  4

  const b = await env.client.workflow.start(checkoutWorkflow, {
    taskQueue: 'checkout',
    workflowId: uid('order-B'),
    args: ['B'],
  });
  await b.signal(confirmPayment);
  await b.result();
  assert.deepEqual(steps('B'), ['reserve', 'fraud-check', 'charge']); // new path
  await worker.stop();
});
```

4	The in-flight order took the **new** path — and replay stayed deterministic.

```
test('deprecatePatch (v3) only after all pre-patch orders finished', async () => {
  const v2History = await historyOf('v2');
  await Worker.runReplayHistory({ workflowsPath: code('v3') }, v2History); // ok
  const v1History = await historyOf('v1');
  await assert.rejects(
    // too early
    Worker.runReplayHistory({ workflowsPath: code('v3') }, v1History),  5
    (err: Error) => err.name === 'DeterminismViolationError',
  );
});
```

5	Deprecating while v1 orders still run would break them: wait until they finish.

What the tests prove (`test/versioning.test.ts`, all passing):

Test	Verified behaviour
replay tests catch an unsafe change before it reaches production	A real v1 history replays on v2; on the unsafe version it throws `DeterminismViolationError`.
deploy mid-order: the order takes the new path when it gets there	Order A (started on v1, waiting) finishes on v2 **with** the fraud check; new order B too.
deprecatePatch (v3) only after all pre-patch orders finished	v2 histories replay on v3; v1 histories do not.

Watch out: A common misconception: "`patched()` keeps in-flight workflows on the old path." It only does so for history that already passed that line. A workflow still waiting *before* it will run the new code when it gets there, as the test shows. If in-flight executions must not get the new step, route them by version explicitly or use Worker Versioning.

18. Testing

18.1 Setup: Node's test runner + tsx

No extra framework needed. Add `"test": "node --import tsx --test test/*.test.ts"` to package.json. The time-skipping test server is downloaded automatically on the first run.

18.2 Unit-test activities

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { MockActivityEnvironment } from '@temporalio/testing';
import { chargeCard } from '../src/activities';

test('rejects negative amounts', async () => {
  const env = new MockActivityEnvironment({ attempt: 1 });
  await assert.rejects(env.run(chargeCard, 'o-1', -5), /positive/);
});
```

18.3 Test workflows with time skipping

This is the real test suite of the Part VI order service (all tests pass; the 2-minute approval timeout completes in about 3 seconds):

test/order.test.ts (excerpt)

```
// Run with: node --import tsx --test test/*.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { Worker } from '@temporalio/worker';
import { WorkflowFailedError } from '@temporalio/client';
import { ApplicationFailure } from '@temporalio/common';
import { orderWorkflow, approveOrder, getStatus } from '../src/workflows';
import type { OrderInput } from '../src/shared';

let env: TestWorkflowEnvironment;
const calls: string[] = [];

// Mocked activities: record calls instead of touching real systems
const activities = {
  reserveInventory: async (_id: string, sku: string) => {
    calls.push(`reserve:${sku}`);
    if (sku === 'NONE') {
      throw ApplicationFailure.nonRetryable('Out of stock', 'OutOfStock');
    }
  },
  releaseInventory: async () => { calls.push('release'); },
```

```
  chargePayment: async (id: string) => { calls.push('charge'); return `txn-${id}`; },
  refundPayment: async () => { calls.push('refund'); },
  shipOrder: async (id: string) => { calls.push('ship'); return `TRACK-${id}`; },
  sendEmail: async () => { calls.push('email'); },
};

const order = (over: Partial<OrderInput> = {}): OrderInput => ({
  orderId: `o-${Math.random().toString(36).slice(2, 8)}`,
  customerEmail: 'a@b.com', sku: 'BOOK', quantity: 1, amount: 20, ...over,
});

async function withWorker<T>(fn: () => Promise<T>): Promise<T> {
  const worker = await Worker.create({
    connection: env.nativeConnection,
    taskQueue: 'test',
    workflowsPath: require.resolve('../src/workflows'),
    activities,
  });
  return worker.runUntil(fn);
}

before(async () => { env = await TestWorkflowEnvironment.createTimeSkipping(); });
after(async () => { await env?.teardown(); });

test('small order ships without approval', async () => {
  calls.length = 0;
  const input = order();
  const result = await withWorker(() => env.client.workflow.execute(orderWorkflow, {
    workflowId: input.orderId, taskQueue: 'test', args: [input],
  }));
  assert.equal(result, `TRACK-${input.orderId}`);
  assert.deepEqual(calls, ['reserve:BOOK', 'charge', 'ship', 'email']);
});

test('large order auto-cancels after 2 min without approval', async () => {
  calls.length = 0;
  const input = order({ amount: 5000 });
  const started = Date.now();
  const result = await withWorker(() => env.client.workflow.execute(orderWorkflow, {
    workflowId: input.orderId, taskQueue: 'test', args: [input],
  }));
  assert.equal(result, 'CANCELLED');
  assert.ok(Date.now() - started < 30_000, '2-minute timer should be skipped');
  assert.deepEqual(calls, ['email']);
});

test('large order ships after approval signal', async () => {
  calls.length = 0;
  const input = order({ amount: 5000 });
  await withWorker(async () => {
    const handle = await env.client.workflow.start(orderWorkflow, {
      workflowId: input.orderId, taskQueue: 'test', args: [input],
    });
    await env.sleep('30 seconds');                          // advance the test clock
    assert.equal(await handle.query(getStatus), 'AWAITING_APPROVAL');
    await handle.signal(approveOrder, 'manager');
    assert.equal(await handle.result(), `TRACK-${input.orderId}`);
  });
});
```

npm test

```
# tests 4, pass 4, fail 0
#  - small order ships without approval
#  - large order auto-cancels after 2 min without approval
#  - large order ships after approval signal
#  - out-of-stock order fails without compensation
```

18.4 Replay tests (catch non-determinism)

```
import { test } from 'node:test';
import { Connection, Client } from '@temporalio/client';
import { Worker } from '@temporalio/worker';

// Export real histories:  temporal workflow show -w order-123 --output json > h.json
test('new code replays old history', async () => {
  const client = new Client({ connection: await Connection.connect() });
  const history = await client.workflow.getHandle('order-123').fetchHistory();
  await Worker.runReplayHistory(
    { workflowsPath: require.resolve('../src/workflows') }, history);
  // throws DeterminismViolationError if the change is incompatible
});
```

18.5 The real-world examples test suite

All 15 scenarios in this guide live in `temporal-examples/` with one test file each. Every test file starts its own time-skipping server (the schedule test uses the real dev server) and one worker:

```
// test/helpers.ts
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { Worker, WorkerOptions } from '@temporalio/worker';

export interface RunningWorker {
  stop(): Promise<void>;
}

// Runs a worker in the background for the duration of a test file
export async function startWorker(  1
  env: TestWorkflowEnvironment,
  taskQueue: string,
  workflowsPath: string,
  activities: object,
  extra: Partial<WorkerOptions> = {},  2
): Promise<RunningWorker> {
  const worker = await Worker.create({
    connection: env.nativeConnection,
    taskQueue,
    workflowsPath,
    activities,
    ...extra,
  });
  const running = worker.run();
  return {
    async stop() {
      worker.shutdown();
      await running;
    },
  };
}
```

1 One background worker per test file, stopped in `after()`.

2 Per-test worker options (e.g. `maxCachedWorkflows`).

```ts
let seq = 0;
export const uid = (prefix: string) =>
  `${prefix}-${Date.now().toString(36)}-${seq++}`;

// Walks ActivityFailure -> ApplicationFailure -> ... to the original error
export function rootCause(err: unknown): Error | undefined {  3
  let e = err instanceof Error ? err : undefined;
  while (e?.cause instanceof Error) e = e.cause;
  return e;
}
```

3 Failures are nested (Workflow → Activity → Application); tests assert on the root.

```
npm test        # node --import tsx --test --test-concurrency=3 test/*.test.ts
# tests 35, pass 35, fail 0   (about 30 s on a laptop)
npm run format  # prettier; npm run typecheck: tsc --noEmit
```

Test file	Tests	Technique worth copying
`payments`	3	Fake gateway that fails on demand; count calls
`travel-search`	2	Measure wall time to prove parallelism
`shopping-cart`	3	Signals, queries and updates against one handle
`free-trial`	2	`env.sleep('5 days')`, then signal
`reports`	1	`createLocal()` for features needing a full server
`loan-application`	5	Slow fake to trigger a real timeout
`video-transcoding`	2	Crash once, assert resume point; cancel mid-run
`onboarding`	2	Describe child executions by derived IDs
`loyalty`	1	Wait for the new run ID after continue-as-new
`email-campaign`	1	Track max concurrency in the fake provider
`trip-booking`	2	Journal of side effects; flaky compensation
`expense-approval`	4	Advance time past each escalation deadline
`kyc-verification`	2	Count polls; deadline between attempts
`document-review`	2	Complete an activity from "outside"
`versioning`	3	Replay real histories; swap code mid-execution

19. Observability & Debugging

19.1 Web UI

- Filter executions by type, status, ID or search attributes.
- Event history (timeline / compact / JSON), inputs and outputs of every activity.
- Pending activities with attempt count and last failure — first place to look when something seems stuck.
- Send signals, cancel, terminate and reset from the UI.

19.2 Logging

```
// workflows: replay-aware, includes workflowId/runId automatically
import { log } from '@temporalio/workflow';
log.info('Charging', { amount });

// activities: includes activity metadata automatically
import { log } from '@temporalio/activity';
log.warn('Slow gateway', { ms: 3200 });

// worker process: route SDK logs to your logger (winston/pino adapter possible)
Runtime.install({ logger: new DefaultLogger('INFO', (entry) => console.log(entry)) });
```

19.3 Metrics & tracing

```
import { Runtime } from '@temporalio/worker';
Runtime.install({
  telemetryOptions: {
    metrics: { prometheus: { bindAddress: '0.0.0.0:9464' } },  // scrape /metrics
  },
});
// Tracing: @temporalio/interceptors-opentelemetry adds spans for client, workflow
// and activity calls. Key metrics: schedule-to-start latency, task slots used,
// activity failures, workflow task failures.
```

19.4 Debugging checklist

Symptom	Likely cause / fix
Workflow stays Running, nothing happens	No worker on that task queue/namespace. Check the UI "Workers" tab and the task queue name spelling.
Activity attempt count keeps rising	Activity throws each time. Read "last failure" in the UI; add nonRetryable errors or fix the bug.
WorkflowTaskFailed repeated in history	Workflow code threw a plain Error, or non-determinism after a deploy. Fix code and redeploy; it resumes.
`DeterminismViolationError`	Code change without `patched()`. Roll back, or patch, or reset the execution.
Bundling error: Module not found 'fs'/'crypto'	A Node-only package imported (directly or transitively) from workflows.ts. Move to activity, use `import type`.
ActivityFailure: TimeoutFailure START_TO_CLOSE	Activity slower than `startToCloseTimeout`, or worker died mid-activity.

20. Deployment

20.1 Pre-bundle workflow code

Bundling at worker startup takes seconds and needs webpack at runtime. For production, bundle at build time:

```
// scripts/bundle.ts
import { bundleWorkflowCode } from '@temporalio/worker';
import { writeFile } from 'fs/promises';
import path from 'path';

async function bundle() {
  const { code } = await bundleWorkflowCode({
    workflowsPath: require.resolve('../src/workflows'),
  });
  await writeFile(path.join(__dirname, '../workflow-bundle.js'), code);
}
bundle();

// worker.ts (production)
const worker = await Worker.create({
  ...options,
  workflowBundle: { codePath: require.resolve('../workflow-bundle.js') },
});
```

20.2 Dockerfile

```
# glibc base image required by the Temporal worker - do NOT use alpine
FROM node:22-bookworm-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build && npx tsx scripts/bundle.ts && npm prune --omit=dev
ENV NODE_ENV=production
CMD ["node", "lib/worker.js"]
```

20.3 Connecting to Temporal Cloud

```
// API key authentication
const connection = await NativeConnection.connect({    // Connection.connect for clients
  // e.g. <region>.<cloud>.api.temporal.io:7233
  address: process.env.TEMPORAL_ADDRESS!,
  tls: true,
  apiKey: process.env.TEMPORAL_API_KEY!,
});

// mTLS authentication
import fs from 'fs';
const connection = await NativeConnection.connect({
  address: 'my-ns.a1b2c.tmprl.cloud:7233',
  tls: { clientCertPair: {
    crt: fs.readFileSync(process.env.TEMPORAL_CLIENT_CERT_PATH!),
    key: fs.readFileSync(process.env.TEMPORAL_CLIENT_KEY_PATH!),
  } },
});
// namespace: 'my-ns.a1b2c' in Worker.create / new Client(...)
```

Watch out: Never commit API keys or certificates. Load them from environment variables or a secrets manager.

20.4 Scaling & operations

- Scale workers horizontally; watch **schedule-to-start latency** to know when you need more.
- Tune `maxConcurrentActivityTaskExecutions` to what your downstream systems can handle.
- Deploy with rolling restarts; `shutdownGraceTime` lets activities finish.
- Keep payloads small (< 2 MB each; aim for KBs). Store files in S3/DB and pass keys.
- Encrypt sensitive payloads with a custom **PayloadCodec** passed as `dataConverter: { payloadCodecs: [...] }` to both Client and Worker.

- Separate task queues for workloads with different scaling or resource needs.

Part VI — End-to-End Project: Order Service

21. Building an Order-Processing Service

A complete, runnable service tying together everything: an Express HTTP API uses the Temporal Client; a Worker runs an order workflow with manual approval for large orders, customer cancellation, status queries, retries on a flaky payment gateway and saga compensation.

Endpoint	Temporal operation
`POST /orders`	`client.workflow.start(orderWorkflow)`
`GET /orders/:id`	`handle.query(getStatus)` + `handle.describe()`
`POST /orders/:id/approve`	`handle.signal(approveOrder, approver)`
`POST /orders/:id/cancel`	`handle.signal(cancelOrder)`
`GET /orders/:id/result`	`handle.result()`

21.1 Install

```
mkdir order-service && cd order-service && npm init -y
npm install @temporalio/client @temporalio/worker @temporalio/workflow \
            @temporalio/activity express
npm install -D typescript tsx @types/node @types/express @temporalio/testing
# tsconfig.json: the one from Chapter 4.2 (module/moduleResolution "nodenext")
```

21.2 Shared definitions

```
// src/shared.ts
export const TASK_QUEUE = 'orders';

export interface OrderInput {
  orderId: string;
  customerEmail: string;
  sku: string;
  quantity: number;
  amount: number;          // in dollars
}

export type OrderStatus =
  | 'RECEIVED' | 'AWAITING_APPROVAL' | 'RESERVED' | 'CHARGED'
  | 'SHIPPED' | 'CANCELLED' | 'FAILED';
```

21.3 Activities

```
// src/activities.ts
import { activityInfo, ApplicationFailure, log } from '@temporalio/activity';

// Fake inventory for the demo (per worker process)
const stock = new Map<string, number>([['BOOK', 10], ['LAPTOP', 2]]);  1

export async function reserveInventory(orderId: string, sku: string, qty: number) {
  const available = stock.get(sku) ?? 0;
  if (available < qty) {
    throw ApplicationFailure.nonRetryable(`Out of stock: ${sku}`, 'OutOfStock');  2
  }
  stock.set(sku, available - qty);
  log.info('Inventory reserved', { orderId, sku, qty, left: available - qty });
}
```

1. Demo state lives in the worker process. In production this is a database call — activities are exactly where I/O belongs.
2. A **business** error: retrying won't help, so it is non-retryable with a *type* (`OutOfStock`) the workflow can inspect.

```
export async function releaseInventory(orderId: string, sku: string, qty: number) {
  stock.set(sku, (stock.get(sku) ?? 0) + qty);
  log.info('Inventory released', { orderId, sku, qty });
}

export async function chargePayment(orderId: string, amount: number): Promise<string> {
  const { attempt } = activityInfo();  3
  if (attempt < 3) {                        // simulate a flaky gateway
    log.warn('Payment gateway timeout (simulated)', { orderId, attempt });
    throw new Error('Payment gateway timeout');  4
  }
  log.info('Payment charged', { orderId, amount, attempt });
  return `txn-${orderId}`;  5
}
```

3. `activityInfo()` exposes the attempt number, workflow ID and heartbeat details of the current execution.
4. A plain `Error` is treated as transient and retried with the workflow's retry policy — attempts 1 and 2 fail, attempt 3 succeeds.
5. Real payment calls must be **idempotent**: pass `orderId` as the gateway's idempotency key so a retry never charges twice.

```
export async function refundPayment(txnId: string) {
  log.info('Payment refunded', { txnId });
}

export async function shipOrder(orderId: string): Promise<string> {
  return `TRACK-${orderId.toUpperCase()}`;
}

export async function sendEmail(to: string, message: string) {
  log.info('EMAIL', { to, message });
}
```

21.4 Workflow

```
// src/workflows.ts
import {
  proxyActivities, defineSignal, defineQuery, setHandler, condition, log,
  ApplicationFailure, CancellationScope,
} from '@temporalio/workflow';
import type * as activities from './activities';  1
import type { OrderInput, OrderStatus } from './shared';

const {
  reserveInventory, releaseInventory, chargePayment, refundPayment, shipOrder, sendEmail,
} = proxyActivities<typeof activities>({  2
  startToCloseTimeout: '30 seconds',
  retry: { initialInterval: '1 second', backoffCoefficient: 2, maximumAttempts: 5 },  3
});
```

1. **Type-only** import: the workflow sandbox never loads activity code, only its signatures.
2. Typed proxies. Calling `chargePayment()` schedules an activity task; the call returns when a worker has run it.
3. Retries at 1s, 2s, 4s, 8s — this is why the flaky gateway eventually succeeds without any retry code.

```
export const approveOrder = defineSignal<[string]>('approveOrder');  4
export const cancelOrder  = defineSignal('cancelOrder');
export const getStatus    = defineQuery<OrderStatus>('getStatus');

const APPROVAL_LIMIT = 1000;

export async function orderWorkflow(input: OrderInput): Promise<string> {
  let status: OrderStatus = 'RECEIVED';
  let approvedBy: string | undefined;
  let cancelled = false;

  setHandler(approveOrder, (user) => { approvedBy = user; });
  setHandler(cancelOrder, () => { cancelled = true; });
  setHandler(getStatus, () => status);  5
```

4. Message definitions are exported so the API can call `handle.signal(approveOrder, ...)` with full type checking.
5. Register handlers **before the first await** so no early signal or query is missed.

```
  // 1. Large orders need a human
  if (input.amount > APPROVAL_LIMIT) {
    status = 'AWAITING_APPROVAL';
    const decided = await condition(() => approvedBy !== undefined || cancelled,  6
                                    '2 minutes');
    if (!decided || cancelled) {
      status = 'CANCELLED';
      await sendEmail(input.customerEmail, `Order ${input.orderId} was not approved`);
      return 'CANCELLED';
    }
    log.info('Order approved', { approvedBy });
  }

  // 2. Saga: each step registers its undo
  const compensations: Array<() => Promise<void>> = [];
  const checkCancelled = () => {
    if (cancelled) throw ApplicationFailure.nonRetryable('Cancelled by customer');
  };
```

6. Durable wait: survives worker restarts and costs nothing. Returns `false` if 2 minutes pass without approval or cancellation.

```
  try {
    await reserveInventory(input.orderId, input.sku, input.quantity);
    status = 'RESERVED';
    compensations.unshift(() => releaseInventory(input.orderId, input.sku,  7
                                                 input.quantity));
    checkCancelled();  8

    const txnId = await chargePayment(input.orderId, input.amount);
    status = 'CHARGED';
    compensations.unshift(() => refundPayment(txnId));
    checkCancelled();

    const tracking = await shipOrder(input.orderId);
    status = 'SHIPPED';
    await sendEmail(input.customerEmail, `Order ${input.orderId} shipped: ${tracking}`);
    return tracking;
  } catch (err) {
    status = cancelled ? 'CANCELLED' : 'FAILED';
    await CancellationScope.nonCancellable(async () => {  9
      for (const undo of compensations) await undo();
      await sendEmail(input.customerEmail,
                      `Order ${input.orderId} ${status.toLowerCase()}`);
    });
    if (cancelled) return 'CANCELLED';
    throw err;  10
  }
}
```

7	Saga step: after a step succeeds, register its undo. `unshift` makes compensations run in **reverse** order.
8	Cancellation checkpoint between steps: throws a non-retryable failure so the `catch` block rolls back.
9	Cleanup must finish even if the workflow itself is being cancelled.
10	Re-throwing an `ApplicationFailure` marks the execution **FAILED**; the client sees `WorkflowFailedError` (the API maps it to 422).

21.5 Worker

```
// src/worker.ts
import { NativeConnection, Worker } from '@temporalio/worker';
import * as activities from './activities';
import { TASK_QUEUE } from './shared';

async function run() {
  const connection = await NativeConnection.connect({
    address: process.env.TEMPORAL_ADDRESS ?? 'localhost:7233',
  });
  const worker = await Worker.create({
    connection,
    namespace: process.env.TEMPORAL_NAMESPACE ?? 'default',
    taskQueue: TASK_QUEUE,
    workflowsPath: require.resolve('./workflows'),
    activities,
  });
  console.log('Order worker started');
  await worker.run();
}
run().catch((err) => { console.error(err); process.exit(1); });
```

21.6 HTTP API

```ts
// src/api.ts
import express, { Request, Response, NextFunction } from 'express';
import {
  Connection, Client, WorkflowNotFoundError, WorkflowFailedError,
} from '@temporalio/client';
import { orderWorkflow, approveOrder, cancelOrder, getStatus } from './workflows';
import { TASK_QUEUE, OrderInput } from './shared';

// Express 5 forwards rejected promises from async handlers to the error middleware

async function main() {
  const connection = await Connection.connect({
    address: process.env.TEMPORAL_ADDRESS ?? 'localhost:7233',
  });
  const client = new Client({ connection, namespace: 'default' });  // 1
  const app = express();
  app.use(express.json());
```

1 Create **one** client per process and reuse it for every request.

```ts
  app.post('/orders', async (req, res) => {
    const input: OrderInput = { orderId: `ord-${Date.now()}`, ...req.body };
    const handle = await client.workflow.start(orderWorkflow, {
      taskQueue: TASK_QUEUE,
      workflowId: `order-${input.orderId}`,  // 2
      args: [input],
    });
    res.status(202).json({ workflowId: handle.workflowId });  // 3
  });

  app.get('/orders/:id', async (req, res) => {
    const handle = client.workflow.getHandle(req.params.id);
    const desc = await handle.describe();
    // Queries are answered by a worker; without one they would hang, so cap the wait
    const status = await connection
      .withDeadline(Date.now() + 5_000, () => handle.query(getStatus))  // 4
      .catch(() => 'UNKNOWN (no worker available)');
    res.json({ workflowId: req.params.id, status, execution: desc.status.name });
  });
```

2 Business ID as Workflow ID: a duplicate start for the same order is rejected by the server.

3 `start()` returns as soon as the server has recorded the workflow — hence `202 Accepted`.

4 Queries are answered by a worker. Without one they would hang, so the call is bounded to 5 s.

```
  app.post('/orders/:id/approve', async (req, res) => {
    await client.workflow.getHandle(req.params.id)
      .signal(approveOrder, req.body.approver ?? 'admin');  5
    res.sendStatus(202);
  });

  app.post('/orders/:id/cancel', async (req, res) => {
    await client.workflow.getHandle(req.params.id).signal(cancelOrder);
    res.sendStatus(202);
  });

  app.get('/orders/:id/result', async (req, res) => {
    const result = await client.workflow.getHandle(req.params.id).result();  6
    res.json({ result });
  });
```

5 Signals are stored by the server even when no worker is running; the workflow receives it when a worker picks it up.

6 `result()` long-polls until the workflow completes (or throws if it failed).

```
  app.use((err: unknown, _req: Request, res: Response, _next: NextFunction) => {
    if (err instanceof WorkflowNotFoundError) {
      return res.status(404).json({ error: 'not found' });
    }
    if (err instanceof WorkflowFailedError) {          // business failure, not a crash  7
      let cause: Error | undefined = err.cause;         // unwrap ActivityFailure etc.
      while (cause?.cause instanceof Error) cause = cause.cause;
      return res.status(422).json({ status: 'FAILED', reason: cause?.message });
    }
    res.status(500).json({ error: String(err) });
  });

  app.listen(3000, () => console.log('API on http://localhost:3000'));
}
main().catch((err) => { console.error(err); process.exit(1); });
```

7 A failed workflow is a business outcome: unwrap the cause chain and return 422 with the real reason, e.g. `Out of stock: LAPTOP`.

21.7 Run the whole system

```
temporal server start-dev          # terminal 1 - Temporal Service + UI :8233
npx tsx src/worker.ts              # terminal 2 - Worker
npx tsx src/api.ts                 # terminal 3 - HTTP API :3000
# no CLI installed? terminal 1 alternative: npx tsx scripts/dev-server.ts (3.4a)
```

On Windows PowerShell use `curl.exe` (not the `curl` alias) or any REST client.

```
# A) Small order - completes automatically (payment succeeds on attempt 3)
curl -X POST localhost:3000/orders -H "Content-Type: application/json" \
  -d '{"customerEmail":"ann@example.com","sku":"BOOK","quantity":1,"amount":25}'
# {"workflowId":"order-ord-1727512345678"}

curl localhost:3000/orders/order-ord-1727512345678
# {"status":"SHIPPED","execution":"COMPLETED", ...}

# B) Large order - waits for approval
curl -X POST localhost:3000/orders -H "Content-Type: application/json" \
  -d '{"customerEmail":"bob@example.com","sku":"LAPTOP","quantity":1,"amount":2400}'
curl localhost:3000/orders/order-ord-...            # "AWAITING_APPROVAL"
curl -X POST localhost:3000/orders/order-ord-.../approve \
  -H "Content-Type: application/json" -d '{"approver":"manager-jo"}'
curl localhost:3000/orders/order-ord-.../result     # {"result":"TRACK-ORD-..."}

# C) Out of stock - non-retryable failure, workflow FAILED, nothing to compensate
curl -X POST localhost:3000/orders -H "Content-Type: application/json" \
  -d '{"customerEmail":"cy@example.com","sku":"LAPTOP","quantity":5,"amount":500}'

# D) Approval timeout - large order with no approval for 2 minutes -> CANCELLED
```

21.8 What to observe

- **Worker logs**: two simulated payment timeouts, then success on attempt 3 — retries with exponential backoff, no code for it.
- **Web UI**: the chargePayment activity shows attempts; pending approval orders show a timer and signal events.
- **Durability**: create a large order, stop the worker, approve via the API, restart the worker — the signal was buffered and the order completes.
- **Compensation**: send `/cancel` while an order is in progress to see refundPayment then releaseInventory run (reverse order).
- **Failures are business results**: `GET /orders/:id/result` for the out-of-stock order returns **422** with `"reason":"Out of stock: LAPTOP"` (root cause unwrapped from the ActivityFailure).
- **Queries need a worker**: with the worker stopped, `GET /orders/:id` answers after 5 s with `"status":"UNKNOWN (no worker available)"` instead of hanging (`connection.withDeadline`).

21.9 Next steps for this project

- Replace the in-memory stock with a real database via the activity factory (6.2).
- Add tests with `TestWorkflowEnvironment` for the approval timeout (18.3).
- Pre-bundle and containerize the worker (20.1-20.2); point it at Temporal Cloud (20.3).
- Add a `CustomerId` search attribute and list a customer's orders.

Appendices

A. CLI Reference

```
# Server
temporal server start-dev [--db-filename f.db] [--ui-port 8233] [--namespace ns]

# Workflows  (-w = --workflow-id)
temporal workflow start     --type orderWorkflow --task-queue orders -w order-1 \
                            --input '{"orderId":"1","sku":"BOOK","quantity":1}'
temporal workflow execute   ...same flags...          # start and wait for result
temporal workflow list      --query "ExecutionStatus='Running'"
temporal workflow describe  -w order-1
temporal workflow show      -w order-1 [--output json]  # event history
temporal workflow result    -w order-1
temporal workflow signal    -w order-1 --name approveOrder --input '"alice"'
temporal workflow query     -w order-1 --type getStatus
temporal workflow update execute -w cart-1 --name applyCoupon --input '"SAVE10"'
temporal workflow cancel    -w order-1
temporal workflow terminate -w order-1 --reason "bad input"
temporal workflow reset     -w order-1 --type LastWorkflowTask
temporal workflow stack     -w order-1           # where is the code blocked?

# Task queues, schedules, namespaces
temporal task-queue describe --task-queue orders   # see connected workers
temporal schedule list | describe | trigger | delete --schedule-id X
temporal operator namespace create --namespace orders
```

B. Troubleshooting Quick Reference

Error / symptom	Fix
`Connection refused 127.0.0.1:7233`	Server not running, or wrong `address`. Start `temporal server start-dev`.
`Namespace default is not found`	Create it, or pass the right `namespace` to both Worker and Client.
`WorkflowExecutionAlreadyStartedError`	Workflow ID already running. Use a new ID, or `workflowIdConflictPolicy: 'USE_EXISTING'`.
`Activity function X is not registered`	Worker's `activities` object lacks that function, or you are running an old worker.
`Workflow type X not found`	Workflow not exported from `workflowsPath`, or a different worker is polling the same queue.
Workflow bundling fails with Node module errors	Non-deterministic import in the workflow file tree; use `import type` / move code to activities.
Worker fails to start on Alpine	Use a Debian/Ubuntu (glibc) based image.
Query hangs / times out	Queries are answered by a worker. Start one, or bound the call with `connection.withDeadline(Date.now() + 5000, ...)`.
`Failed to load native binding` (webpack / swc) on Windows	Seen with `@swc/core` 1.16.13 (native cache permission check). Pin `"overrides": { "@swc/core": "1.16.2" }` in package.json.
Workflow task fails with `TypeError`, workflow keeps running	Plain errors in workflow code fail the *task*, which is retried. Fix and redeploy; e.g. `RetryState` must be imported from `@temporalio/common`.
Deadline passed but no `TimeoutFailure`	Expired between retries: check `ActivityFailure.retryState === 'TIMEOUT'` (see 16.3).
`ts-node` / `ts-jest` crash with TypeScript 7	TS 7 has no JS API. Use `tsx` and `node --test` or Vitest.
`TMPRL1100 / nondeterminism`	See Chapter 17 — patch or reset.

C. Glossary

Term	Definition
Activity	Function that performs side effects; retried by Temporal.
Command	Instruction from workflow code to the server (schedule activity, start timer).
Continue-As-New	Completing a run and atomically starting a new one with fresh history.
Determinism	Same history ⇒ same commands. Required for replay.
Event History	Durable, ordered record of an execution's events.
Local Activity	Short activity executed within a workflow task on the same worker.
Namespace	Isolation boundary for workflows, queues and settings.
Replay	Re-executing workflow code against history to rebuild state.
Run ID	Unique ID of one run of a workflow ID (changes on continue-as-new/retry).
Saga	Pattern of undoing completed steps when a later step fails.
Schedule	Server-side cron-like trigger that starts workflows.
Search Attribute	Indexed field on an execution usable in list queries.
Signal / Query / Update	Messages to write / read / read-write a running workflow.
Task Queue	Queue by which the server dispatches tasks to workers.
Worker	Process hosting workflow and activity code, polling task queues.

Further resources

- Docs: docs.temporal.io/develop/typescript
- API reference: typescript.temporal.io
- Samples: github.com/temporalio/samples-typescript
- Free courses: learn.temporal.io (Temporal 101 / 102 / Versioning, TypeScript track)
- Community: community.temporal.io and the Temporal Slack